

**Multi-Detection of Root-Knot Nematode with Yolov11-Seg and
Classification with RepVGG-CA**



UNDERGRADUATE FINAL YEAR PROJECT

**Evan Aditya Chandra
2022390002**

**Computer Science
Faculty of Engineering and Technology
Sampoerna University
Jakarta
2026**

**Multi-Detection of Root-Knot Nematode with Yolov11-Seg and
Classification with RepVGG-CA**



UNDERGRADUATE FINAL YEAR PROJECT

**Evan Aditya Chandra
2022390002**

Supervisor:

A handwritten signature in black ink, appearing to be "Ariana Tulus Purnomo", is positioned above the supervisor's name.

Ariana Tulus Purnomo, M.T., Ph.D.

19 June 2026

EXAMINER BOARD APPROVAL

Name of Student : Evan Aditya Chandra
Student ID Number : 2022390002
Study Program : Computer Science
Faculty : Faculty of Engineering and Technology
Final Year Project Title : Multi-Detection of Root-Knot Nematode with
YOLOv11-Seg and Classification with REPVGG-CA

has been successfully defended in front of the Board of Examiner in partial fulfillment of the requirements for Bachelor of Computer Science in Computer Science, Faculty of Engineering and Technology, in Jakarta, 02 June 2026.

Board of Examiners

Examiner 1



Panji Nursetia Dharma, S.Kom., Ph.D.

Examiner 2



Gilang Raka Rayuda Dewa, M.Sc., Ph.D.

Supervisor 1



Ariana Tulus Purnomo, M.T., Ph.D.

DECLARATION OF ORIGINALITY

I, Evan Aditya Chandra 2022390002, here acknowledge that this submission entitled “Multi-Detection of Root-Knot Nematode with YOLOv11-Seg and Classification with REPVGG-CA” is my own work with the guidance of my supervisor. I also truly acknowledge that it contains no materials previously published or written by another person, or any other educational institution, except where due acknowledgment is made in the Final Year Project. Any contribution made to the research by others is explicitly acknowledged in the Final Year Project.

Jakarta, 02 June 2026.



Evan Aditya Chandra

EXCLUSIVE RIGHT STATEMENT

I, Evan Aditya Chandra, hereby grant to our school, Sampoerna University, the non-exclusive right to archive, reproduce, and distribute my Final Year Project entitled “Multi-Detection of Root-Knot Nematode with YOLOv11-Seg and Classification with REPVGG-CA” in whole or in parts, whether in printed or electronic formats. I acknowledge that I retain exclusive rights of my Final Year Project by using all or parts of it in future work or output, such as articles, books, software, and information system.

Jakarta, 02 June 2026.



Evan Aditya Chandra

ACKNOWLEDGEMENT

I would like to express my deepest gratitude to all the individuals and institutions who have supported and contributed, directly or indirectly, to the successful completion of this Final Year Project.

First and foremost, I am immensely thankful to my supervisor, Ariana Tulus Purnomo, for their invaluable guidance, insightful criticism, and unwavering encouragement throughout this entire process, ensuring the technical rigor and feasibility of the research.

Jakarta, 02 June 2026



Evan Aditya Chandra

ABSTRACT

Full Name : Evan Aditya Chandra
Study Program : Computer Science
Title : Multi-Detection Of Root-Knot Nematode With YOLOv11-Seg And Classification With RepVGG-CA
Supervisor : Ariana Tulus Purnomo

Agriculture faces threats from plant-parasitic nematodes, particularly Root-Knot Nematodes, causing devastating yield reductions. Traditional diagnostic methods rely on manual microscopic examination, which is time-intensive, subjective, and error-prone. Current systems perform generic object detection without species identification or classify exclusively on manually pre-cropped images. This research develops an automated two-stage deep-learning pipeline capable of simultaneous detection and species-level classification directly from raw microscopic images. The methodology utilizes 244 microscopic images containing 278 nematode instances, partitioned into a 70:20:10 ratio for training, validation, and testing. To address limited sample sizes and class imbalance, geometric and photometric data augmentation protocols are applied. The first stage employs a proposed YOLOv11n-seg-DCA model operating on 320x320 inputs. This architecture integrates a Deformable Curvature Attention module and a curvature-weighted Dice loss function to localize nematodes. The generated masks isolate regions of interest that are cropped, resized to 224x224, and processed by a customized RepVGG-CA classification network. This classifier leverages a Coordinate Attention mechanism and structural re-parameterization to evaluate morphological details. The isolated detection phase achieved 99.30% mean Average Precision at an intersection over union threshold of 0.50 for bounding boxes and masks, with a 100.00% recall rate. The isolated RepVGG-CA classifier yielded 89.29% test accuracy. When fully integrated, the pipeline demonstrated an accuracy of 88.00%, precision of 86.67%, recall of 88.19%, and F1 score of 87.27%. Processing full-field images requires a total inference latency of 138.14 milliseconds per image, equating to 7.24 frames per second. This framework provides a computational foundation that accelerates manual diagnostics.

Keywords: *Root-Knot Nematodes, object detection, classification, YOLO, RepVGG*

TABLE OF CONTENTS

EXAMINER BOARD APPROVAL.....	iii
DECLARATION OF ORIGINALITY.....	iv
EXCLUSIVE RIGHT STATEMENT.....	v
ACKNOWLEDGEMENT.....	vi
ABSTRACT	vii
TABLE OF CONTENTS	viii
LIST OF FIGURES	xi
LIST OF TABLES.....	xiii
LIST OF FORMULAS	xiv
LIST OF ABBREVIATIONS	xvi
CHAPTER I.....	1
1.1. Background	1
1.2. Problem Formulation	2
1.3. Research Objectives	4
1.4. Significance of the Study.....	5
1.5. Limitation of Study	6
1.6. Organization of the report.....	6
CHAPTER II.....	7
2.1. Research Context.....	7
2.1.1. Nematode	7
2.1.1.1 Plant-Parasitic Nematodes (PPNs)	7
2.1.1.2. Root-Knot Nematodes (Meloidogyne spp.)	8
2.1.2. Object Detection.....	9

2.1.3. Image Classification.....	9
2.1.4. Computer Vision	9
2.1.5. Machine Learning.....	10
2.1.5.1. Deep Learning	10
2.1.5.2. Convolutional Neural Networks (CNN).....	11
2.1.5.3. Visual Geometry Group (VGG).....	11
2.1.5.4. Reparametrized Visual Geometry Group (RepVGG)	12
2.2. Previous Work.....	13
2.2.1. Detection and Counting of Root-Knot Nematodes using YOLO Models.....	13
2.2.2. Deep Learning Models for Automatic Identification of Plant-Parasitic Nematode.....	14
2.2.3. Applying Faster R-CNN for Object Detection on Malaria Images	16
2.2.4. A-RepVGG: Research on Classification Algorithms based on Deep Learning and Wood CT Images.....	18
CHAPTER III	21
3.1. Overview of Research Design	21
3.2. Data Preparation	21
3.3. Model Architecture.....	27
3.3.1. Object Detection Model: YOLOv11-Seg.....	27
3.3.2. Classification Model: RepVGG-CA	33
3.4. Training Strategy	45
3.4.1. Yolov11-seg-DCA.....	45
3.4.2. RepVGG-CA	48
3.5. Evaluation Metrics	50

CHAPTER IV	56
4.1. Object Detection Model (Yolov11-seg-DCA).....	56
4.1.1. Training.....	56
4.1.2. Result and Comparison	60
4.2. Classification Model (RepVGG-CA)	68
4.2.1. Training.....	68
4.2.1. Result and Comparison	71
4.3. Combined Pipeline	83
CHAPTER V	88
5.1. Conclusion	88
5.2. Suggestions	89
REFERENCES.....	91
BIOGRAPHY AUTHOR.....	94

LIST OF FIGURES

Figure 1 Pun et al's Nematode Detection.....	13
Figure 2 Shabrina et al's results	15
Figure 3 Hung & Carpenter's Two Stage pipeline	17
Figure 4 Example in the dataset for multiple nematode in a single image (multi-nematode).....	23
Figure 5 Example in the dataset for a single nematode	24
Figure 6 Yolov11n-seg-DCA model architecture	28
Figure 7 Head block for Yolov11n-seg-DCA model architecture	33
Figure 8 Original RepVGG-A0 Architecture	35
Figure 9 RepVGG-CA Architecture	35
Figure 10 RepVGG block during training	41
Figure 11 RepVGG block during inference	42
Figure 12 Yolov11-seg-DCA Train Box-loss	56
Figure 13 Yolov11-seg-DCA Train Segmentation-loss.....	57
Figure 14 Yolov11-seg-DCA mAP50 (box).....	58
Figure 15 Yolov11-seg-DCA mAP50-95 (box).....	59
Figure 16 Object detection model complexity	66
Figure 17 Object detection inference time comparison	66
Figure 18 Object detection FPS comparison.....	67
Figure 19 RepVGG-CA loss graph	69
Figure 20 RepVGG-CA accuracy graph	70
Figure 21 Test accuracy comparison original and tuned parameters	72
Figure 22 Test accuracy comparison of the models	73
Figure 23 RepVGG-CA confusion matrix	77
Figure 24 RepVGG-A0 confusion matrix.....	78
Figure 25 EfficientNetV2B0 confusion matrix.....	79
Figure 26 Classification model complexity	80
Figure 27 Classification inference time comparison.....	81

Figure 28 Classification FPS comparison 81



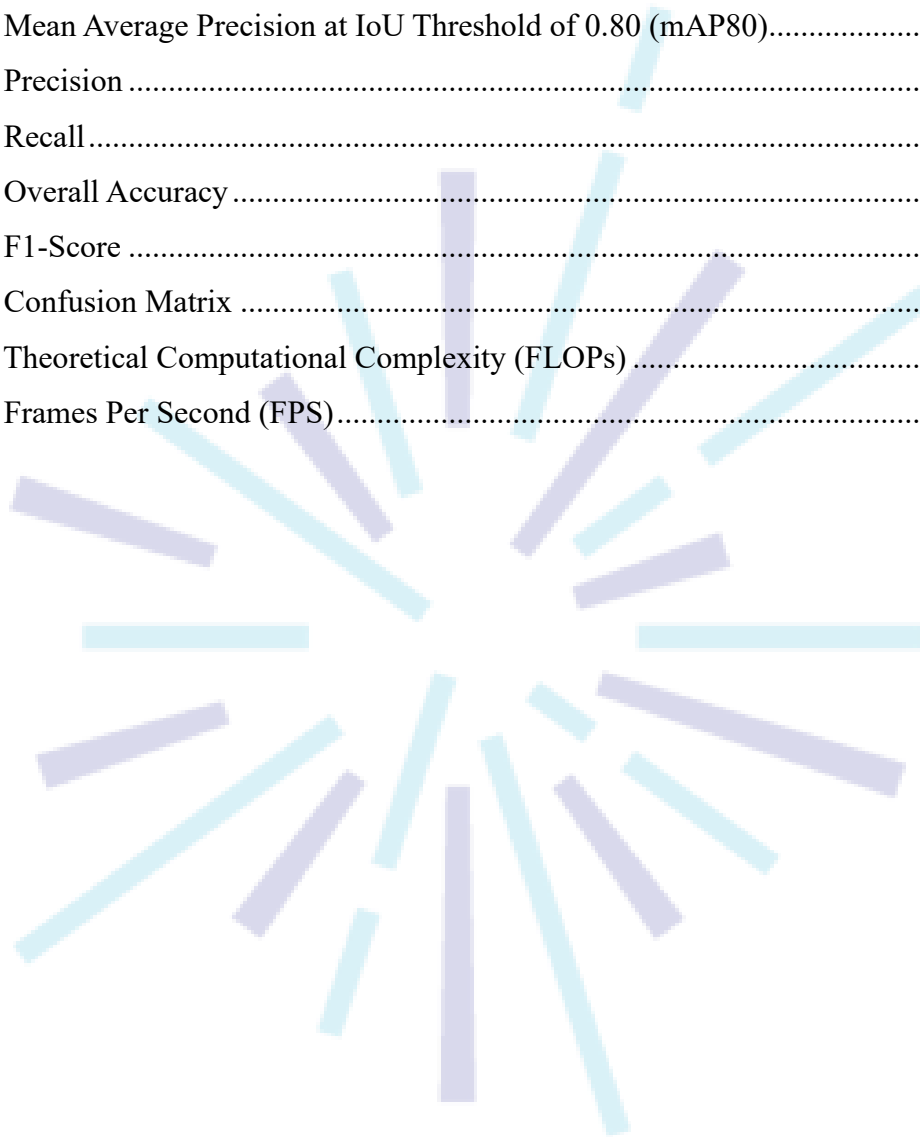
LIST OF TABLES

Table 1 Summary of Literature Review	19
Table 2 Dataset Object Detection Image Count	22
Table 3 Dataset Classification Inference Count	22
Table 4 Augmentation Table for Object Detection.....	25
Table 5 Augmentation Table for Classification.....	26
Table 6 Parameter Summary for Yolov11n-seg-DCA.....	47
Table 7 HyperParameter Summary for RepVGG-CA	50
Table 8 Bounding Box Performance Comparison for Optimizer.....	61
Table 9 Segmentation Performance Comparison for Optimizer.....	61
Table 10 Bounding Box Performance Comparison	63
Table 11 Segmentation Performance Comparison	63
Table 12 Parameter Summary for Tuned Parameters.....	71
Table 13 Meloidogyne Graminicola Performance Metrics	75
Table 14 Meloidogyne Incognita Performance Metrics.....	75
Table 15 Pipeline classification result.....	83
Table 16 Full pipeline results	84
Table 17 Pipeline inference time.....	85
Table 18 Pipeline Troughput.....	85

LIST OF FORMULAS

(1) First Intermediate Feature Map (C3k2 Block)	29
(2) Second Intermediate Feature Map (C3k2 Block).....	29
(3) Final Aggregated Output Tensor (C3k2 Block).....	29
(4) Initial Offset Prediction (Deformable Offsets)	30
(5) Discrete Laplacian Operator	30
(6) Final Curvature Attention Weight.....	31
(7) Softmax Coefficients And Final Instance Segmentation Mask	32
(8) Coordinate Attention (CA) Mechanism.....	32
(9) Global Channel Statistics (Global Average Pooling)	36
(10) Fully Connected Module	36
(11) Final Predicted Probability (Softmax).....	37
(12) Repvgg Block Multi-Branch Computational Topology	37
(13) Identity Shortcut Branch Extension	38
(14) Total Pre-Activation Output (BN Fusion)	39
(15) Batch Normalization Fusion Into Convolutional Weights	40
(16) Zero-Padded 1x1 Fused Weight Matrix.....	40
(17) Zero-Padded Identity BN-Fused Weights.....	40
(18) Final Merged Weight Matrix And Bias	40
(19) Final Inference Output Feature Map	41
(20) Horizontal Statistics (Coordinate Attention)	43
(21) Vertical Statistics (Coordinate Attention).....	43
(22) Shared Compressed Embedded Representation	43
(23) Horizontal Attention Weights	44
(24) Vertical Attention Weights	44
(25) Full Coordinate Attention Map.....	44
(26) Recalibrated Output Feature Map	44
(27) Cosine Annealing Learning Rate.....	46
(28) Composite Loss Function	46

(29) Curvature-Weighted Dice Loss	46
(30) Laplacian Operator	46
(31) Cosine Decay Learning Rate Schedule	49
(32) Smoothed Cross-Entropy Loss	49
(33) Mean Average Precision at IoU Threshold of 0.50 (mAP50).....	50
(34) Mean Average Precision at IoU Threshold of 0.50-0.95 (mAP50-95).....	51
(35) Mean Average Precision at IoU Threshold of 0.80 (mAP80).....	52
(36) Precision	52
(37) Recall.....	52
(38) Overall Accuracy	53
(39) F1-Score	53
(40) Confusion Matrix	53
(41) Theoretical Computational Complexity (FLOPs)	54
(42) Frames Per Second (FPS).....	54



LIST OF ABBREVIATIONS

Abbreviation	Full Form
A-RepVGG	Attention-enhanced Re-parameterized Visual Geometry Group
AI	Artificial Intelligence
AP	Average Precision
BB	Bounding Box
BN	Batch Normalization
CA	Coordinate Attention
CABI	Centre for Agriculture and Bioscience International
CE	Cross-Entropy
CIoU	Complete Intersection over Union
CNN	Convolutional Neural Network
COCO	Common Objects in Context
CoAtNet-0	Convolution and Attention Network-0
CPU	Central Processing Unit
CSP	Cross Stage Partial
CT	Computed Tomography
CVF	Computer Vision Foundation
CVPRW	Computer Vision and Pattern Recognition Workshops
DCA	Deformable Curvature Attention
DFL	Distribution Focal Loss
DWConv3	Depthwise Convolution 3×3
ELU	Exponential Linear Unit
EMA	Exponential Moving Average
EPN	Entomopathogenic Nematode
EfficientNetV2	EfficientNet Version 2
EfficientNetV2-B0	EfficientNet Version 2 B0
EfficientNetV2B0	EfficientNet Version 2 B0
EfficientNetV2M	EfficientNet Version 2 Medium
F1-Score	Harmonic Mean of Precision and Recall
FLN	Free-Living Nematode
FN	False Negative
FP	False Positive
FPN	Feature Pyramid Network
FPS	Frames Per Second

Abbreviation	Full Form
GAP	Global Average Pooling
GB	Gigabyte
GFLOP	Giga Floating Point Operation
GHz	Gigahertz
GPU	Graphics Processing Unit
IEEE	Institute of Electrical and Electronics Engineers
IITA	International Institute of Tropical Agriculture
ImageNet	Image Database for Visual Recognition
IoU	Intersection over Union
IPM	Integrated Pest Management
mAP	Mean Average Precision
mAP50	Mean Average Precision at IoU = 0.50
mAP50-95	Mean Average Precision averaged over IoU thresholds from 0.50 to 0.95
MBConv	Mobile Inverted Bottleneck Convolution
NMS	Non-Maximum Suppression
OpenCV	Open Source Computer Vision Library
PAN	Path Aggregation Network
PAN-FPN	Path Aggregation Network–Feature Pyramid Network
PPN	Plant-Parasitic Nematode
R-CNN	Region-based Convolutional Neural Network
RAM	Random Access Memory
ReLU	Rectified Linear Unit
RepVGG	Re-parameterized Visual Geometry Group
RepVGG-A0	Re-parameterized Visual Geometry Group A0
RepVGG-CA	Re-parameterized Visual Geometry Group with Coordinate Attention
ResNeSt	Split-Attention Network
ResNet	Residual Network
ResNet-50	Residual Network with 50 Layers
ResNet101V2	Residual Network 101 Version 2
RKN	Root-Knot Nematode
RKNs	Root-Knot Nematodes
RMSprop	Root Mean Square Propagation
RoI	Region of Interest
RoIs	Regions of Interest
RSM-YOLOv11	RSM-enhanced YOLOv11

Abbreviation	Full Form
SE	Squeeze-and-Excitation
SGD	Stochastic Gradient Descent
SGDR	Stochastic Gradient Descent with Warm Restarts
SharedMLP	Shared Multi-Layer Perceptron
SiLU	Sigmoid Linear Unit
SPPF	Spatial Pyramid Pooling Fast
TN	True Negative
TP	True Positive
TPU	Tensor Processing Unit
VGG	Visual Geometry Group
VGG16	Visual Geometry Group 16
VGG19	Visual Geometry Group 19
VGGNet	Visual Geometry Group Network
ViT	Vision Transformer
VRAM	Video Random Access Memory
YOLO	You Only Look Once
YOLOv2	You Only Look Once Version 2
YOLOv3	You Only Look Once Version 3
YOLOv5	You Only Look Once Version 5
YOLOv7	You Only Look Once Version 7
YOLOv8	You Only Look Once Version 8
YOLOv11	You Only Look Once Version 11
YOLOv11-seg	YOLOv11 Segmentation
YOLOv11-seg-DCA	YOLOv11 Segmentation with Deformable Curvature Attention
YOLOv11n-seg	YOLOv11 Nano Segmentation
YOLOv11n-seg-DCA	YOLOv11 Nano Segmentation with Deformable Curvature Attention

CHAPTER I

INTRODUCTION

1.1. Background

Agriculture forms the central pillar of human existence, as it offers the necessary mechanism to achieve food security worldwide. Being under continuous attack from various vectors, such as pests and diseases, this vital organ of human society triggers considerable crop losses and endangers the stability of global food networks. For staple crops, including maize, rice, and potatoes, the yield decrease attributed to pest activity is substantial, at 31, 37, and 40 percent, respectively (Oerke, 2006). Plant-parasitic nematodes (PPNs) are among the most economically significant threats to the global food industry. They are obligate biotrophs, microscopic and soil-dwelling, that have developed to feed exclusively on living plant tissue. They cause a destructive, silent, and unannounced attack on the roots of almost all significant crops such as cereals, legumes, tubers, fruits, and vegetables (Jones et al., 2013). They can cause a relentless toll on the health of crops in agricultural soils, and their minute size often means they appear without notice until significant damage has already been caused.

PPNs also impair the ability of the plant to absorb water and other nutrients by feeding on plant roots. The disruption is experienced in a spectrum of unspecific symptoms of above-ground nature, including stunting, wilting, and chlorosis (yellowing of the leaves), most of which are commonly confused with nutritional or water deficiency and thus make diagnosis difficult (Williamson & Hussey, 1996). International measurements have found that PPNs lead to losses of around 80-118 billion of crops every year, which is a significant and probably underestimated number, given that a large proportion of farmers are not equipped with the necessary tools to detect PPNs (Shah & Mahamood, 2017). This threat is disproportionately affecting smallholders in developing countries, where agriculture is a key

component of the economy, and access to sophisticated diagnostic tools is often limited.

The present-day trend indicates an increased risk, and global climate change is a severe exacerbator of the threat posed by PPNs. According to recent climate forecasts, an increase in soil temperature could not only shorten the life cycles of species but also expand the geographical range of thermophilic nematode species to previously hostile agricultural areas (Dutta & Phani, 2023). This implies that agriculturalists in temperate regions will soon face new challenges from nematode species that are typically restricted to tropical and subtropical regions, and therefore, an increased need to monitor with greater accuracy than ever before.

Artificial intelligence algorithms have emerged as a highly promising solution to such diagnostic dilemmas. Previously studied research has demonstrated that Convolutional Neural Networks (CNNs) can be utilized to automate PPN analysis in two significant ways. To start with, YOLO (You Only Look Once) models have been utilized to detect and count nematodes in complex microscopic images with high precision, thereby facilitating the detection of objects (Pun et al., 2023). Second, image classification is performed by models, such as EfficientNet and ResNet, on pre-cropped images of individual nematodes to differentiate between different classifications with high accuracy (Shabrina et al., 2023).

1.2. Problem Formulation

Despite the significant impact of plant-parasitic nematodes (PPNs) on world farming, their exact diagnosis has not been done properly because of the limitations of the traditional methods. The standard workflow is a manual process that has been made with a few changes. The sampling of the fields is conducted first; the nematodes are secondly removed from the soil and organic waste through laboratory extraction procedures (Coyne, Nicol, & Claudius-Cole, 2014). Lastly, a nematologist looks at the samples using a microscope to analyze the nematode's morphological features, such as tail-tip structure, body size, or style of the stylet (Eisenback, 1985). The complexity of the species also contributes to the problem

of classification through the eyes; genetically different species may have similar morphology, resulting in mismanagement advice.

It is often necessary to have hours of specialized expertise, which can make the whole process too time-intensive. Additionally, it is subjective, as considerable differences in numbers were observed among the analysts, and is prone to human error, especially in cases where the duration of microscopic examination causes fatigue. Bottlenecks resulting from these inefficiencies may lead to delays of one week, thereby hindering farmers' ability to act proactively rather than reactively. This is entirely incongruous with the need to make decisions quickly and based on facts required by modern Integrated Pest Management (IPM) concepts (Sikora & Fernandez, 2005).

With the emergence of deep learning, the possibility of automating the PPN analysis process has been introduced, and two different directions have been examined in recent studies. The first is dedicated to the recognition and classification of PPNs, where neural networks like YOLO are trained with the aim of identifying and listing the presence of 7 different objects and classifying them as PPNs or non-PPNs (Pun et al., 2023). The second is classification, where models such as EfficientNet are trained on single, separated nematode images to distinguish species with a high level of accuracy (Shabrina et al., 2023).

However, in spite of the success of both methodologies, it demonstrates the presence of a research gap. The detection models, while capable of detecting images with multiple objects, do not provide the species or genus-level classification necessary for targeted management. Conversely, the classification models can accurately classify single images of nematodes to their genus (requiring a pre-processed image of a single nematode). Also, the two processes of object detection and classification have not yet been integrated. This highlights a critical problem because existing datasets are tailored for one task or the other; there is no universal, formatted dataset designed for the end-to-end task of both detecting and specifically classifying nematodes.

Based on the problem of the previous studies, it can be summarized in the following points:

- While highly effective within their designated scope, existing classification models are specifically optimized for single-subject inputs and are not intended to process uncropped images containing multiple nematodes. Because field samples typically feature several specimens in close proximity, a gap remains for solutions that can handle complex, multi-subject fields of view.
- Conversely, current detection models successfully locate nematodes in uncropped images, fulfilling their primary objective of localization. However, because their architectural focus is on detection rather than classification, they leave an opportunity for further development to support the simultaneous classification required during real-world sampling.
- While significant advancements have been made in both detection and classification independently, there remains an opportunity to integrate these functions into a unified system capable of processing raw images containing multiple nematodes.

1.3. Research Objectives

Therefore, based on sub-section 1.2, the problem this research aims to solve is a unified model that combines the strengths of both approaches. A research gap exists for a multi-class AI model that can accurately detect and differentiate between multiple PPN species from a single, uncropped or cropped microscopic image. The primary objective is to create a single system that first locates all nematode instances in an image and then assigns a specific species to each detection. To achieve this goal, the following objectives have been established:

- To train a multi-detection model. An object detection architecture will be trained to locate nematodes by using an object detection architecture on an existing data with a target mAP50-mask of 99.5% (Pun et al, 2023).

- To train a classification model. A classification model will be trained to differentiate the species of nematodes by optimizing a classification architecture on the output of the detection model with target accuracy of 98% (Shabrina et al, 2023).
- To develop a pipeline that will combine detection and classification models. The project will develop a workflow that combines the two models into one, enabling the detection and classification of nematodes with minimal human involvement.

1.4. Significance of the Study

This study is important because it validates a computational framework for accurately extracting and differentiating the visually similar microscopic organisms. This study shows that deep learning algorithms can correctly locate and classify multiple species of root-knot nematodes in a single image, making a solid algorithmic basis for further developments in agricultural diagnostics. Proves that attention mechanisms can overcome the small and cluttered nature of the nematode datasets to detect the morphological changes in nematodes, using Deformable Curvature Attention and Coordinate Attention.

This study is an example of the benefits of changing from general pest detection to species-specific intelligence. Available computational methods generally only detect nematode presence rather than classify to a species level, a result of which this research has shown to be achievable from raw images. Given that different species need completely different approaches to management (e.g., *Meloidogyne incognita* versus *Meloidogyne graminicola*), generic detection is not enough for modern agriculture. The results of this study show that data can be used to narrow down the species classification to the individual species level and will be able to inform the selection of resistant crop varieties and targeted soil treatments.

Overall, this project serves as a technical landmark for better food security and environmental sustainability worldwide. Root-knot nematodes are a serious problem for crop staples and can be difficult to manage, so fast and accurate identification is essential. This is not a field-deployable system, but it nevertheless

serves as a proof of concept for future systems. This work offers the framework to identify pests that are targeted, allowing the shift to proactive Integrated Pest Management (IPM) that reduces the ecological impact of agriculture by using targeted chemical treatments instead of generalised ones.

1.5. Limitation of Study

This study is focused on creating a multi-detection and classification model and testing its outcomes to be able to categorise PPNs based on an image input. The study will make use of an already existing dataset of images of two *Meloidogyne* species, *M. incognita* and *M. graminicola*, at juvenile and adult stages.

There are a number of limitations to this research. To begin with, the effectiveness of the model depends on the quality, diversity, and quantity of the training data, as the dataset is quite limited in number. Second, it is a computational study and does not involve actual sampling, extraction of nematodes, or preparation of slides in physical form. The study starts with images as the input. Third, the paper only covers two *Meloidogyne* species: *M. incognita* and *M. graminicola*. Lastly, the 64-bit cloud environment, which is used in the model training, utilizes an Intel Xeon CPU, 2 GHz, and 30 GB of RAM, and two NVIDIA Tesla T4 GPUs, each with 16 GB of dedicated VRAM.

1.6. Organization of the report

This report is structured into five main chapters. In chapter 1, the topic of the research is given, the context of the problem is outlined, and the objectives and significance of the study are outlined. In the second chapter, the literature review is conducted, background information on the topic and literature reviews of previous papers are provided. In Chapter 3, the general overview of the approach, data preprocessing, model architecture, training strategy, and metrics to be used to evaluate the outcomes are described. Chapter 4 presents the Results and Discussion, and Chapter 5 offers a Conclusion, summarizing the study's findings.

CHAPTER II

LITERATURE REVIEW

2.1. Research Context

2.1.1. Nematode

Nematodes are microscopic and rounded worms and constitute one of the most plentiful animal groups on Earth (van den Hoogen et al., 2019). They also measure between 0.3mm and 3mm in length and can live in a wide variety of ecosystems, including both the marine and the microscopic film of the soil water (Oliveira et al., 2011). Nematodes have a series of different ecological functions; most of them are free-living organisms that consume bacteria, fungi, or other microorganisms, and they are vital to the ecological processes like decomposition (Yeates et al., 1993). Some species, e.g., entomopathogenic nematodes (EPNs), are useful in agriculture since they naturally regulate insect pests, which makes them the appropriate biological insecticides (Shah and Mahamood, 2017). However, some species of the nematode phylum are also known to be parasitic to plants, invertebrates, and vertebrates, thus posing threats to global agriculture and the health of people (Oliveira et al., 2011).

2.1.1.1 Plant-Parasitic Nematodes (PPNs)

Plant-parasitic Nematodes are a subtype of nematodes that feed on and attack plants. PPNs are a great menace to the world's agriculture, with heavy losses of crops being experienced worldwide (Jones et al., 2013). These nematodes have also developed to rely completely on a host plant to reproduce and survive (Shah & Mahamood, 2017). The characteristic that makes them unique is the stylet, a sharp, needle-like mouth part that helps them to pierce the plant cells and steal their nutrients and thus divert the energy stored in the plant to the survival of the nematode (Coyne et al., 2014; Williamson and Hussey, 1996). This parasitism harms the root system because it hampers the ability of the plant to absorb water and essential

nutrients (Shah & Mahamood, 2017). In addition, physical forms of damage caused by infecting nematodes enable the introduction of other fungi and bacteria, causing more serious diseases (Dutta & Phani, 2023).

As a result of the destruction of PPNs, infected plants often display non-specific above-ground symptoms such as wilting, yellowing of leaves, and stunted growth, and may be mistaken for water stress or nutrient deficiency (Sikora and Fernandez, 2005). This, in turn, has a major effect on global agriculture and food security (Afzal and Mukhtar, 2024). Out of thousands of PPN species, a few genera contribute to most of the agricultural damage. Root-knot nematodes (*Meloidogyne* spp.), cyst nematodes (*Globodera* spp. and *Heterodera* spp.), and root-lesion nematodes (*Pratylenchus* spp.) are ranked as the most economically devastating nematodes because of their high rates of reproduction and broad host ranges (Jones et al., 2013).

2.1.1.2. Root-Knot Nematodes (*Meloidogyne* spp.)

Root-K Nematodes must establish a permanent parasitic relationship inside a host plant to complete their life cycle. The second-stage juvenile is the infectious stage; it travels through the soil, finds a host root, and enters the root tissue of said host. After that, the juvenile moves to the vascular cylinder, where it injects proteins into nearby plant cells that cause the plant cells to develop into enormous, multinucleate "giant cells." These giant cells function as a nutrient source, nourishing the developing nematode (Williamson & Hussey, 1996).

These giant cells are the distinctive knots on the plant's root system that give the RKN their name. These knots impair the plant's ability to absorb and transport water and other nutrients from the soil by disrupting the vessels of the plant root (Williamson & Hussey, 1996). This disruption is a significant problem, as it manifests with non-specific, above-ground symptoms that are often mistaken for other causes mentioned in the previous section on PPNs (Sikora & Fernandez, 2005).

2.1.2. Object Detection

Object detection is a central computer-vision methodology that determines object locations in an image and localizes them through bounding boxes and classifies them (Zhang et al., 2021). In comparison to elementary classification tasks, it supports a complex scene, heterogeneous objects with varying scale, spatial location, as well as overlaps. Such can be useful for autonomous vehicles, surveillance systems, and medical imaging (More & Bhosale, 2023). In the specific domain of agricultural diagnostics, these exact capabilities are highly valuable for localizing multiple microscopic plant-parasitic nematodes amid cluttered soil debris and complex backgrounds in diagnostic imaging (Pun et al., 2023).

2.1.3. Image Classification

Image classification is the process of labeling a single semantic meaning to a whole image through the extraction of features and the recognition of patterns in the visual content of an image (Zhang et al., 2021). This task is the basis of applications such as photo organization, content moderation, and disease diagnosis based on radiography or other scans. The historical development of this field has gone beyond the original means to the use of advanced convolutional neural networks, and some examples of such architectures are ResNet (More & Bhosale, 2023). Within nematology, image classification forms the necessary foundation for identifying specific species based on subtle but distinct morphological features present in isolated or cropped microscopic images (Shabrina et al., 2023).

2.1.4. Computer Vision

Computer vision is a sub-discipline of artificial intelligence that gives computing machines the ability to perceive and extract information from computer-generated images, videos, and other visual data. It aims to imitate the visual system of the human eye by using a series of functions that mimic visual perception (Szeliski, 2022). Using artificial intelligence models, especially the

deep-learning networks such as convolutional and neural networks (CNNs), computer vision algorithms are capable of identifying objects, classifying images, and extract information from visual inputs (Goodfellow et al., 2016). This technology works as a human eye for machines and allows them to observe and examine their surroundings with one image or several images (Szeliski, 2022; Goodfellow et al., 2016). When applied to microscopic biological analysis, computer vision acts as an automated extension of a nematologist, allowing systems to rapidly process complex field samples to detect and classify organisms.

2.1.5. Machine Learning

Machine learning is a subfield of artificial intelligence that examines approaches through which systems could learn from data and use the learned information to make decisions or predictions. Machine-learning programs are trained on data to identify patterns, which is different compared to traditional programming, whereby the coder creates code based on strict rules. Generalization is the way an AI model can predict results of previously unknown data that are not observed during training (Bishop, 2006). The main argument is that increased exposure to data improves model performance, which in turn increases the model's decision-making abilities (Mitchell, 1997). Machine learning is the basis on which computer vision systems are developed. A typical image-analysis machine-learning workflow consists of: image preprocessing to remove noise, object of interest segmentation against the background, manual extraction of numerical values of size, shape, and texture, and training a classifier to make a prediction based on these values (Zinchuk & Grossenbacher, 2020).

2.1.5.1. Deep Learning

One of the subfields of machine learning is deep learning, which has revolutionized computer-vision studies (Goodfellow et al., 2016). Deep learning uses multilayer neural networks (in contrast to the conventional

machine-learning paradigms, which use selected features and learns data representations at successively higher levels of abstraction (LeCun, Bengio, & Hinton, 2015). The most important benefit of deep learning is automatic feature extraction: as the network is being trained, internal parameters (weights) are changed to minimize error, thus demonstrating complex structures in high-dimensional data directly on raw input (Goodfellow et al., 2016). This enables the hierarchical representations of features, such that initial layers acquire features like simple edges or textures. In later stages, the features are integrated to form more complicated objects and shapes (Goodfellow et al., 2016).

2.1.5.2. Convolutional Neural Networks (CNN)

Convolutional neural networks (CNNs) are a sub-specialization of deep learning designed to handle grid-like data structures, such as images. Whereas traditional machine learning requires humans to extract features by hand, CNNs can find patterns on their own from raw pixel data (Goodfellow et al., 2016). Low-level convolutional layers encode basic features, such as edges and textures, which are later aggregated by higher-level convolutional layers into higher abstractions. As the information moves on with each extra layer, these characteristics are combined into more intricate forms and semantic features (Szeliski, 2022). As such, CNNs are especially useful in undertaking biological activities that require complex or subtle visual representations and are referred to as plant-parasitic nematodes in microscope images (Shabrina et al., 2023).

2.1.5.3. Visual Geometry Group (VGG)

The VGG architecture (VGG16 and VGG19) pioneered by the Visual Geometry Group at Oxford University changed the way deep learning was practiced because it used a series of small (3 3) filters instead of large ones as in the previous models (Simonyan & Zisserman, 2014). The principle here is that when small convolutional layers are stacked, they have

the benefits of more than using one large layer. As an example, two sequential 3×3 layers are capable of perceiving the same receptive field as a single 5×5 filter and adding other nonlinearities. This has two advantages: on the one hand, this method adds more processing steps between inputs and outputs, which makes the network more discriminative in the recognition of complex shapes, and on the other hand, the overall computational cost is lower than when only one large filter is used (Simonyan & Zisserman, 2014).

Structurally, VGG is simple; it is a "single path" network, meaning data flows in a straight line from start to finish. It uses blocks of filters to extract information, moving from simple edges at the beginning to complex textures at the end. However, while this simple, straight-line structure runs fast on computers, it is hard to train. If the network is too deep, the learning signal becomes lost, causing it to learn more slowly and perform worse than modern models that utilize more complex paths (Ding et al., 2021).

2.1.5.4. Reparametrized Visual Geometry Group (RepVGG)

RepVGG is a convolutional neural network architecture proposed by Ding et al. (2021). It performs a decoupling between the multi-branch topology used during training and a simplified, single-path inference model in a process called structural re-parameterization. During the training phase, the network features a more complex, ResNet-inspired architecture that incorporates parallel 1×1 convolutions and identity shortcuts in addition to the core 3×3 convolutional layers. Such a configuration may be viewed as an implicit set of shallow models, taken in large numbers, thereby stabilizing convergence and mitigating the pathological effects of extremely deep networks. The network is, however, reparametrized to remove the computational overhead and memory usage associated with deploying a multi-branch design. Utilizing simple algebra, the parameters from the parallel identity and 1×1 branches are mathematically merged into the main 3×3 kernels, converting the model into a single-path stack of 3×3

convolutions and ReLU operations. This reparameterization enables RepVGG to achieve the high accuracy characteristics of complex multi-branch networks while retaining the superior inference speed and memory efficiency of simple VGG-style plain architectures on hardware devices.

2.2. Previous Work

2.2.1. Detection and Counting of Root-Knot Nematodes using YOLO Models

The research conducted by Pun et al. (2023) created a framework for the identification and counting of root-knot nematodes (RKNs) in a microscopic image shown in figure 1.

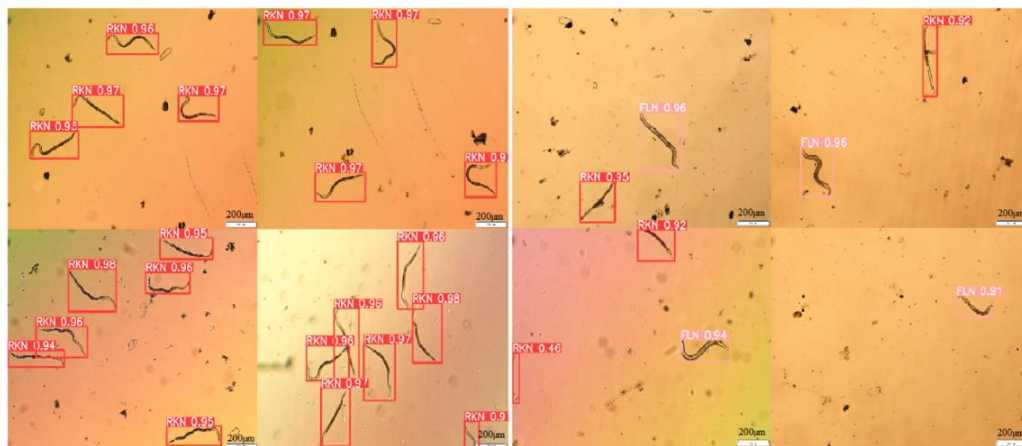


Figure 1 Pun et al's Nematode Detection

The authors tested several variants of the YOLO family of detectors and compared their quality in detecting both RKNs and free-living nematodes (FLNs). The most successful setup achieved a mean average precision (mAP) of 0.97, which is better than previous approaches to this use. One of the key elements of their pipeline was the use of mosaic augmentation, where four different training images are merged into one composite image. This effect has been successfully used to imitate differences in the scale of nematodes that appear when images are taken at different magnifications. The study's main objective is to determine whether a nematode is present in an image, and the high mAP score establishes a reliable baseline and confirms this ability.

The results above highlight the legitimacy of the use of YOLO in detecting nematodes in microscopic imagery. However, the paper also highlights a research gap that persists, namely that its classification was confined to the binary distinction between RKN and FLN, as opposed to a more finely resolved species or genus identification necessary for practical nematode management. In turn, this left the problem of species-level classification unsolved, since the initial purpose was detection and counting. The current study directly fills this gap, as it does not merely limit the pipeline to mere detection; the output of the detection model is further input to the classification model, which identifies the nematode at the species level.

2.2.2. Deep Learning Models for Automatic Identification of Plant-Parasitic Nematode

Shabrina et al. (2023) compared several deep-learning backbones in the classification of plant-parasitic nematodes (PPNs) on microscopic images. The authors selected and cut separate images of nematodes of eleven different genera and provided a comparative evaluation of ResNet101v2, CoAtNet-0, EfficientNetV2B0, and EfficientNetV2M. All backbones were trained with three learning-rate regimes, including Adam, SGD, and RMSprop, and produced a total of twelve experimental settings. The EfficientNetV2B0 and EfficientNetV2M models, trained using RMSprop, achieved the best test accuracy of 97.94% seen in figure 2. Other data-augmentation methods, such as horizontal flipping, Gaussian blurring, and simultaneous brightness-contrast manipulations, were also used to support results on the small dataset. All of these findings indicate that deep-learning algorithms are capable of reliably learning the morphological differences between genera of nematodes in microscopic images, which justifies the classification aspect of the current study.



Figure 2 Shabrina et al's results

Nevertheless, the main weakness of the study conducted by Shabrina et al. is the use of pre-cropped data of single nematode images. Since the work is about classification, as opposed to detection, the authors have not addressed the issue of first locating and separating several nematodes on the same image, a step required for a real-world implementation. The fact that the preparation of such a dataset should be done manually adds significant time cost and lowers the feasibility of the method.

Nevertheless, the paper presents strong arguments for the viability of deep-learning-based classification of nematodes based on the microscopic imaging. The accuracy of the classification pipeline is demonstrated by the 97.94% accuracy reported across eleven genera. Moreover, the effective usage

of the data-augmentation strategies supports their purpose to help address the issue of data scarcity. However, the reliance on manually processed inputs highlights an excellent gap in the literature: the lack of a pipeline capable of detecting and classifying various nematodes in multi-object and complex images.

2.2.3. Applying Faster R-CNN for Object Detection on Malaria Images

The study by Hung & Carpenter (2017) provided a methodological framework that bridges the detection-and-classification paradigm. Their study dealt with the recognition of microscopic biological objects in complex images, a problem that is similar to nematode detection and classification, but in the context of detecting malaria. Using the two-stage detector Faster R-CNN as shown in figure 3, the authors filtered out Plasmodium parasites and classified them into one of four stages of the parasitic life-cycle using infected blood smear images. This approach was clearly aimed at meeting common problems of microscopic image data, including strong class imbalance, a limited number of annotated examples, and variable cell morphologies and dark fields.

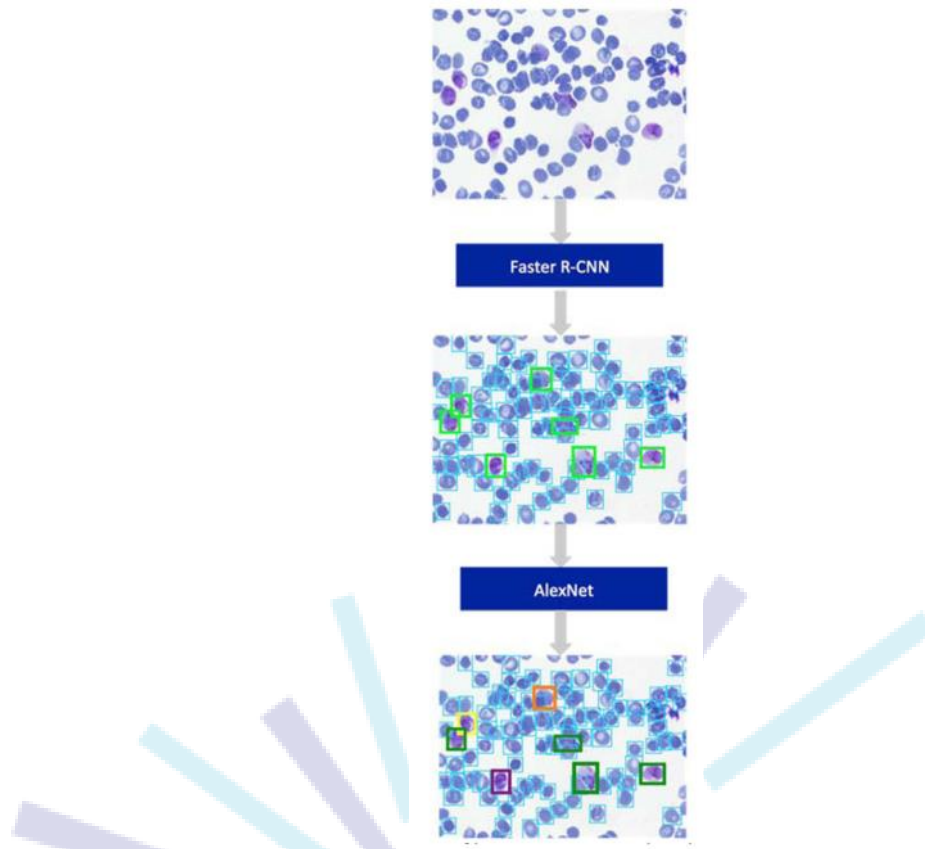


Figure 3 Hung & Carpenter's Two Stage pipeline

The main value of the paper is the empirical experience that the sequential detection-plus-classification pipeline can achieve high diagnostic accuracy and, therefore, the modelling approach that is parallel to the one needed to identify nematodes is valid. However, the direct transfer of the results to the current setting is impeded by the applicability of the particular architectures and the specific domain constraints. In the context of the current paper, Hung & Carpenter's primary limitations are the specific models used and the domain of the application. It was designed for malaria parasites rather than nematodes; although it is in the microscopy field, the details of what the model is trained upon and is searching for are different. This project directly builds upon the methodological standard set by Hung & Carpenter (2017). In the field of PPN identification, it uses the same two-step method.

2.2.4. A-RepVGG: Research on Classification Algorithms based on Deep Learning and Wood CT Images

The study by Zheng and friends focuses on classifying wood species at the microscopic level, which is done by analyzing cross-sectional images from CT scans using RepVGG. The main aim of this study is to create a model that can be used to identify twenty wood species by utilizing structural characteristics, including pore distribution, tubular pore, and cellular organization. This would improve the accuracy and efficiency of wood identification, which is essential in curbing counterfeit products and high quality control in wood-related industries.

The researchers used three extensions to the baseline RepVGG architecture. First, an Adaptive Convolutional Module is an adaptive convolutional kernel modulation that adapts convolutional kernel responses based on input data, thus better capturing fine-grained texture and structure details. Second, a Multi-Scale Dilated Attention mechanism, the features are extracted at varying scales using dilated convolutions, which focus on the salient parts of the image and minimize the computational costs. Third, the output activation mechanism is replaced with ELU instead of ReLU in order to enhance the extraction of features, especially in the negative activation regime. The data consists of tomography (CT) images, augmented with rotation, mirroring, and the addition of Gaussian noise, which is beneficial in model training.

The resulting model achieved a classification accuracy of 99.50 % on the wood CT dataset and also achieved high rates of recall, precision, and F1-scores. It outperformed networks such as ResNet, ResNeSt, and Vision Transformer (ViT) and took a shorter time to train. These results indicate the ability of this model to obtain deep and detailed features and generalize well for classification tasks.

The first limitation of the study is important to address; the research started with a small dataset, as the samples used were of twenty wood species,

and they were scanned only at five different positions of the samples. In order to make this sparse data accessible to a deep network like RepVGG, the authors used a lot of data augmentation, a methodological approach that was also borrowed in this study. The RepVGG architecture achieved an accuracy of 99.50%. While the data-intensive Vision Transformer (ViT) failed to generalize and obtained significantly lower accuracy in comparative tests. The findings suggest that the RepVGG structure can work with such augmented, limited data. Table 1 below describes the summary of the literature review that have been discussed.

Table 1 Summary of Literature Review

Title	Focus	Relevance	Limitation
During sampling, a single image may contain multiple nematode species; therefore, simultaneous multi-species classification within one image is required. Current detection models can locate nematodes in uncropped images, but they cannot classify them by species. As a result, they cannot support species-level identification. (Shabrina et al., 2023).	Nematode Detection & Counting (RKN vs. FLN)	Used and compared YoloV2 through YOLOv7 and achieved map of 97% with YOLOv5 for nematode detection.	Lacked Species-Level Classification (only RKN vs. FLN).
Deep learning models for automatic identification of Plant-parasitic nematode (Pun et al., 2023).	Nematode Classification (11 Genera)	Used and compared four models with three optimizers. Achieved 97.94% test accuracy With EfficientNetV2 for nematode classification.	Lacked the Detection component. Require pre-cropped individual Images
Applying Faster R-CNN for Object Detection on Malaria Images (Hung & Carpenter, 2017).	Malaria Parasite Detection & Classification	Validated a two-stage approach of detection and classification with R-CNN achieved 98%	Different domain (malaria)

Title	Focus	Relevance	Limitation
		Accuracy within the microscopic domain.	
A-RepVGG: Research on Classification Algorithms based on Deep Learning and Wood CT Images	Classifying wood species at the microscopic level	Proved that a model based on RepVGG can achieve good results with a limited dataset that is then augmented.	Different domain (Wood classification)



CHAPTER III

RESEARCH METHODOLOGY

3.1. Overview of Research Design

The proposed system is architecturally designed and can be divided into two computational stages. The first step is the detection of nematodes with the YOLOv11n-seg-DCA model. The instance-segmentation system has exact bounding-box localisation and pixel-level mask prediction, which, as a consequence, results in tight enclosures around the curvilinear bodies of nematodes with the discrimination of cluttered backgrounds. This model is chosen due to the proof that this model can perform well on nematode detection. This step would scan the whole image to detect the position of single nematodes within, which are then processed further. The second step is the classification model, which would take an image of a single nematode and classify the species using the RepVGG-CA architecture. The generated segmented masks of the detection phase are used as input to this classification model. This model is chosen due to proof that the model could perform well with a dataset that is limited in size, which is then augmented.

3.2. Data Preparation

The current study is based on a microscopic dataset as the input, which consists of about 244 pictures representing *Meloidogyne graminicola* and *Meloidogyne incognita* in both singular (example in figure 1 below) and multiple-nematode (example in figure 2 below) configurations. Within the 244 images, there are 278 instances of *Meloidogyne graminicola* and *Meloidogyne incognita*. The dataset uses a split ratio of 70% for training, 20% for validation, and 10% for testing, respectively. The dataset has a slight imbalance of around 3:2 in favor of *Meloidogyne graminicola*.

The current study utilizes a microscopic image dataset containing approximately 244 images of *Meloidogyne graminicola* and *Meloidogyne incognita*. The images include both single-nematode configurations (an example is

shown in Figure 1) and multiple-nematode configurations (an example is shown in Figure 2) .In terms of image count, the dataset is divided using a 70:20:10 ratio for training, validation, and testing, respectively. The distribution of images for each data split is presented in Table 2 for the object detection dataset.

Table 2 Dataset Object Detection Image Count

Class	Training set	Validation set	Testing set
Meloidogyne graminicola	104	30	15
Meloidogyne Incognita	66	19	10

Since a single image may contain more than one nematode, the total number of annotated instances differs from the number of images. Across the 244 images, the dataset contains 278 instances of Meloidogyne graminicola and Meloidogyne incognita. The distribution of images for each data split is presented in Table 3 for the classification dataset

Table 3 Dataset Classification Inference Count

Class	Training set	Validation set	Testing set
Meloidogyne graminicola	119	34	17
Meloidogyne Incognita	75	22	11

The dataset exhibits a slight class imbalance, with approximately a 3:2 ratio favoring Meloidogyne graminicola. To address this, a data-augmentation protocol is established. This protocol limits the production of augmented images of the class that is overrepresented for the training set (M. graminicola) to generating a final set of training that is balanced at 1:1 between the two species. By applying this protocol, the risk of the models overfitting to the majority class of M. graminicola is mitigated. However, class balancing is not applied to the object detection dataset, as the model's primary objective is the detection of nematodes regardless of species.

To prepare the raw image dataset for the detection model, the study employs a specific annotation strategy known as instance segmentation labeling. In comparison to regular bounding boxes, pixel-level mask annotations provide precise boundary delineation of nematode bodies, effectively separating subjects and cluttered backgrounds.

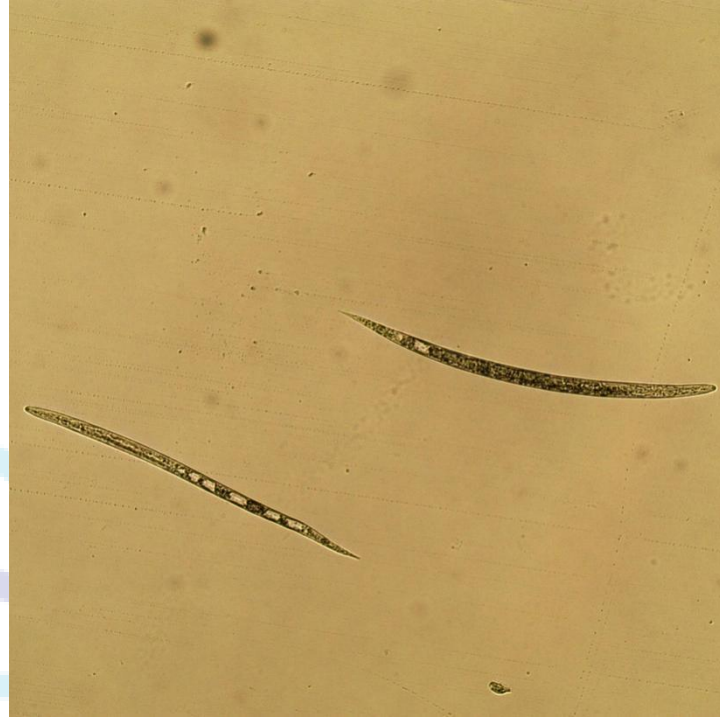


Figure 4 Example in the dataset for multiple nematode in a single image (multi-nematode)



Figure 5 Example in the dataset for a single nematode

After the instance segmentation annotation, the main dataset is partitioned with the aim of testing the performance of the model without bias. The split ratio of 70:20:10 is used, dividing data into training, validation, and testing sets, respectively. The model is trained on the training set, validated during training, with the testing set reserved for final evaluation. The images are resized to 320x320. In order to improve the model even more and avoid overfitting, especially considering the small size of the dataset, the pipeline includes data augmentation techniques. For object detection model data augmentation, such as rotations ($\pm 45^\circ$, $\pm 90^\circ$), flips (vertical and horizontal), and two random photometric augmentation which apply hue shifts ($\pm 2.5^\circ$, $\pm 5^\circ$, $\pm 7.5^\circ$, or $\pm 10^\circ$), brightness adjustments ($\pm 10\%$, $\pm 20\%$, or $\pm 30\%$), and contrast modifications ($\pm 10\%$, $\pm 30\%$, or ± 50) selected randomly from predefined parameter pools. The total after augmentation is shown in Table 4 below.

Table 4 Augmentation Table for Object Detection

Augmentation	Logic	Meloidogyne graminicola	Meloidogyne incognita
ORIGINAL DATA	Input	104	66
Phase 1: Rotations			
Rotations (4 angles)	Original x 4	416	264
Rotation subtotal	Rotated + Original	520	330
Phase 2: Flips			
Flips (2 directions)	Rotations total x 2	1040	660
Flip subtotal	Flips + Rotation total	1560	990
Phase 3: Photometry			
Photometry Augments (2 variations)	Flips total x 2	3120	1980
Total	Photometry + Flips total	4680	2970

The next step is to create a second dataset for the classification phase. Based on the segmentation mask ground truth, individual nematode instances are precisely cropped from the wide-field original images using the predicted mask boundaries. These Regions of Interest (RoIs) form a clean, isolated dataset where background noise is minimized. They are later down-sampled to a fixed resolution (224×224) to fit the input requirements of the RepVGG-CA architecture. Classification augmentation includes random rotations (-180° to 180°) and flips (vertical and horizontal). This creates a specialized classification dataset labeled by species (*M. incognita* vs. *M. graminicola*), enabling the classification step to concentrate only on fine-grained biological features rather than processing the entire microscopic field of view. The total after augmentation, following the protocol to handle imbalance, is shown in Table 5 below.

Table 5 Augmentation Table for Classification

Stage / Operation	Logic / Input Source	Meloidogyne graminicola	Meloidogyne incognita
ORIGINAL DATA	Input	119	75
Phase 1: Rotations			
Rotations (4 angles)	Original x 4 (adjusted for the majority class)	256	300
Rotation subtotal	Rotated + Original	375	375
Phase 2: Flips			
Flips (2 directions)	Rotations total x 2	750	750
Total	Flips + Rotation total	1125	1125

Even though the nematodes would fit better within a rectangular resolution due to their elongated shape, both the YOLOv11n-seg detection model and the RepVGG-CA classification model would use square input resolutions to meet the architectural requirement of convolutional neural networks of homogenous tensor shapes. Modern CNNs, such as YOLO versions, are based on stride downsampling and feature-pyramid networks (FPNs), which avoid changing the height/width ratio when initialised with square inputs (Ultralytics, 2025). For RepVGG-CA, sequential 3×3 convolution blocks expect square receptive fields; rectangular inputs produce ragged tensor dimensions after repeated strides, breaking VGG-style architectural symmetry established since (Simonyan & Zisserman, 2015).

The choice of differing input resolution of 224×224 in RepVGG-CA and 320×320 in YOLOv11n-seg can be explained by the existence of different spatial resolution needs of classification and detection, respectively. Detecting models require higher resolution to retain fine localization information on nematodes in the full-field microscopy images, but classification based on cropped regions of interest (ROIs) by segmentation masks may take advantage of adequate discriminant morphological information at reduced resolutions. Detection networks need pixel-

precise predictions of boundaries to create masking. 320x320 inputs can give 20-80-pixel objects a resolution of C3k2 without significant down-sampling loss (Ultralytics, 2025). On the other hand, models based on classification, like RepVGG, which follows ImageNet standards set by VGGNet, are optimal feature extractors at 224 x 224, where the global context, i.e., body shape and tail structure, prevails over localization (Simonyan & Zisserman, 2015; Ding et al., 2021). This difference of inputs between models would not affect the pipeline as the output of the detection model would not be used directly, but it would be the basis on which the input image would be cropped, then said image would be cropped and resized to 224x224 for the classification model. Though there would be cases when the output of the detection model indicates the optimal crop for the image would not be square. In this case, letterbox padding would be used to preserve the aspect ratio when converting to the square form, therefore removing the distortion, and still being computationally efficient (Redmon & Farhadi, 2018).

3.3. Model Architecture

3.3.1. Object Detection Model: YOLOv11-Seg

YOLOv11n-seg-DCA represents a modified instance segmentation architecture that incorporates the Deformable Curvature Attention (DCA) module within the YOLOv11n-seg detection head. The full architecture can be seen in Figure 3 below.

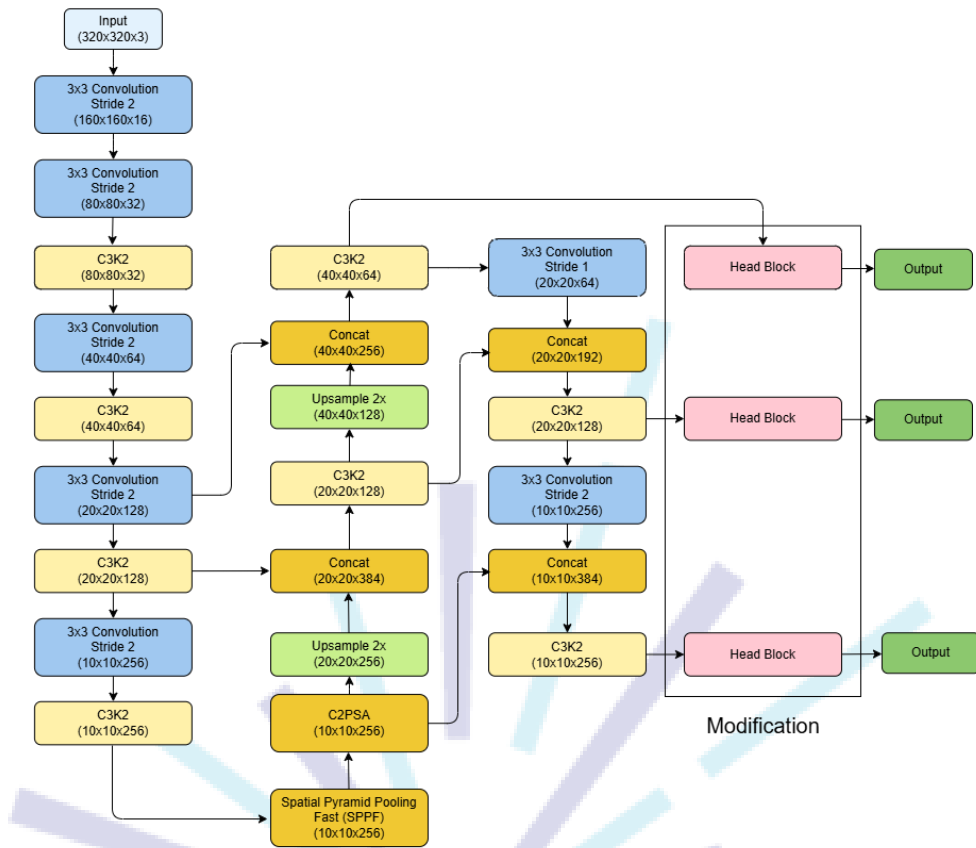


Figure 6 Yolov11n-seg-DCA model architecture

The backbone leverages standard C3k2 blocks, lightweight CSP bottlenecks featuring two 3×3 kernels that achieve significant parameter reduction over conventional C2f blocks through efficient channel splitting and reduced computation while preserving representational capacity (Khanam & Ahmed, 2024). Input images of $320 \times 320 \times 3$ resolution pass through a stem Conv2d layer ($3 \rightarrow 16$ channels, stride=2) followed by two progressive C3k2 stages expanding from $16 \rightarrow 32 \rightarrow 64$ channels, terminated by SPPF (Spatial Pyramid Pooling Fast) for multi-scale feature aggregation.

The neck follows PAN-FPN (Path Aggregation Network with Feature Pyramid Network) architecture, upsampling P5 features ($20 \times 20 \times 512$) and laterally fusing with P4 ($40 \times 40 \times 256$) and P3 ($80 \times 80 \times 128$) backbone outputs through six C3k2 fusion blocks, producing a multi-scale pyramid optimized for

objects spanning 20-80 pixels, typical of nematodes in microscopy fields (Ultralytics, 2025).

Each C3k2 block follows the computational pattern as visualized mathematically in equation 1 till equation 3 below, with x being the first raw input and y as the output.

$$x_1 = \text{Conv}_{1 \times 1}(x) \rightarrow \text{BN} \rightarrow \text{SiLU} \quad (1)$$

$$x_2 = \text{DWConv}_{3 \times 3}(x_1) \rightarrow \text{BN} \rightarrow \text{SiLU} \quad (2)$$

$$y (\text{output}) = \text{Concat}(x_1, x_2) \rightarrow \text{Conv}_{1 \times 1} \rightarrow \text{BN} \rightarrow \text{SiLU} \quad (3)$$

The C3k2 block functions as a lightweight bottleneck designed for efficient feature extraction within the YOLOv11n-seg architecture. The computational sequence begins with Equation 1, where the initial raw input feature map, denoted as x , is processed through a 1×1 convolutional layer ($\text{Conv}_{1 \times 1}$). This operation primarily serves to adjust the channel dimensions of the input. The output is immediately subjected to Batch Normalization (BN), denoted as the first arrow ($\rightarrow$) in figure 1 to stabilize the training process by standardizing the layer inputs, and a Sigmoid Linear Unit (SiLU) activation function is applied, denoted as the second arrow ($\rightarrow$) in figure 1 to introduce non-linearity. This complete sequence yields the first intermediate feature map, x_1 . Subsequently, Equation 2 takes this intermediate tensor and applies a 3×3 depthwise convolution ($\text{DWConv}_{3 \times 3}$) to efficiently capture spatial patterns across individual channels.

Similar to the first step, this output is normalized via Batch Normalization, denoted as the first arrow ($\rightarrow$) in figure 2 and activated using a SiLU function, denoted as the second arrow ($\rightarrow$) in figure 2 to produce the second intermediate tensor, x_2 . Finally, as defined in Equation 3, the block culminates by utilizing a concatenation operation (Concat) to merge the intermediate feature maps x_1 and x_2 along their channel axis. This combined tensor is then fused using a concluding 1×1 convolution denoted as the first arrow ($\rightarrow$) in figure 3. Followed by standard Batch Normalization, denoted as the second arrow ($\rightarrow$) in figure 3 and SiLU activation, denoted as the third

arrow ($\rightarrow$) in figure 3, yielding the final aggregated output tensor (y) which exits the C3k2 block .

The detection head replaces standard decoupled head convolutions with our DCA module positioned before prototype generation and segmentation branches. The head module can be seen in Figure 4 below. DCA Module comprises three sequentially coupled components executed per spatial location across P3/P4/P5 pyramid levels. The process begins by predicting deformable offsets. A lightweight 3×3 convolution predicts 18×2 offset values ($\Delta x, \Delta y$) per anchor relative to fixed $3 \times 3 \times 2$ grid positions with equation 4 below.

$$\Delta p_n = \text{Conv}_{3 \times 3}(x; \theta_{\text{offset}}), p_n = p_0 + \Delta p_n, n = 1, \dots, 18 \quad (4)$$

Equation 4 above defines the initial offset prediction step of the Deformable Curvature Attention (DCA) module. To adapt the network's receptive field to the curvilinear bodies of nematodes, a lightweight 3×3 convolution parameterized by learnable weights (θ_{offset}) is applied to the incoming feature map (x). This operation predicts 18 paired coordinate offset values, represented mathematically as (Δp_n), which correspond to specific spatial shifts in the horizontal and vertical directions for each sampled anchor point. These predicted offsets (Δp_n) are subsequently added to the original, fixed spatial coordinates of a standard $3 \times 3 \times 2$ reference grid, denoted as p_0 , where the index n ranges from 1 to 18 to account for all spatial positions within the receptive field.

The resulting sum produces the deformed sampling coordinates (p_n) By dynamically shifting these sampling points away from the rigid p_0 grid, the module effectively aligns its feature extraction process with the specific morphological contours of the target objects Secondly, the model employs curvature attention weighting. The importance of specific features is modulated by the Laplacian of the ground-truth mask, curvature attention weighting $\nabla^2 M$ calculated with equation 5 below:

$$\nabla^2 M(i, j) = M(i + 1, j) + M(i - 1, j) + M(i, j + 1) + M(i, j - 1) - 4M(i, j) \quad (5)$$

To further refine the feature extraction process, Equation 5 applies a discrete Laplacian operator to the ground-truth segmentation masks, represented by the matrix (M). This operation generates a curvature attention weight ($\nabla^2 M(i, j)$) for a specific target pixel located at spatial coordinates (i, j) . The Laplacian measures the spatial rate of intensity change at this central pixel by subtracting four times its mask value ($4M(i, j)$) from the sum of the mask values of its four immediate spatial neighbors: the pixel to the right ($M(i + 1, j)$), the pixel to the left ($M(i - 1, j)$), the pixel below ($M(i, j + 1)$), and the pixel above ($M(i, j - 1)$). Because the absolute magnitudes of this Laplacian response peak at anatomical landmarks characterized by high curvature, such as the head, tail corners, and body undulations of a nematode. This mechanism modulates feature importance by explicitly assigning higher attention weights to the most biologically critical boundary regions

Absolute values $|\nabla^2 M(i, j)|$ peak at anatomical landmarks: head/tail corners, body undulations, remaining near-zero along straight segments. Said curvature weights are computed with equation 6 below:

$$w(i, j) = 1 + \alpha |\nabla^2 M(i, j)|, \alpha = 0.5 \quad (6)$$

Following the computation of the discrete Laplacian, Equation 6 calculates the final curvature attention weight ($w(i, j)$) for a given pixel coordinate (i, j) . This weight is determined by adding a baseline value of 1 to the product of a scaling hyperparameter, alpha, and the absolute value of the previously calculated Laplacian magnitude ($|\nabla^2 M(i, j)|$). The hyperparameter alpha is empirically set to 0.5 to control the influence of the curvature penalty, ensuring that the model does not disproportionately prioritize high-curvature areas at the expense of overall boundary continuity. By adding the baseline of 1, the equation guarantees that even straight boundary segments with near-zero Laplacian responses retain a minimum baseline weight, while the high-curvature regions receive amplified attention during the segmentation process.

Following feature weighting, the architecture utilizes a prototype-based segmentation head. A 32-channel prototype head generates mask prototypes P

$\in \mathbb{R}^{H \times W \times 32}$, matched to pixel embeddings $E \in \mathbb{R}^{H \times W \times 32}$ via softmax coefficients with equation 7 below.

$$C = \text{Softmax}(E \cdot P^T), \widehat{M} = C \cdot P \quad (7)$$

The first part of the equation calculates the softmax coefficients, represented by the matrix C , by applying a Softmax function to the dot product of the pixel embeddings (E) and the transposed mask prototypes (P^T). The pixel embeddings E contain high-resolution spatial features extracted by the network, while the mask prototypes P act as generalized, learnable pattern templates for the nematode shapes. In the second part of the equation, the final instance segmentation mask ($\widehat{M}$) is reconstructed by computing the matrix multiplication of the softmax coefficients C and the mask prototypes P . This operation linearly combines the learned prototype templates according to the spatially aware coefficients, enabling precise, pixel-level delineation of the nematode instances. Coordinate Attention follows DCA for spatial recalibration, decomposing global context into horizontal/vertical components with equation 8 below:

$$\begin{aligned} z_h &= \frac{1}{W} \sum_{w=1}^W x(h, w, :), z_w = \frac{1}{H} \sum_{h=1}^H x(h, w, :) \\ z &= \text{Concat}(z_h, z_w); [a_h, a_w] = \text{SharedMLP}(z) \\ A &= \sigma(a_h) \otimes \sigma(a_w); y = x \otimes A \end{aligned} \quad (8)$$

Equation 8 describes the Coordinate Attention (CA) mechanism, which performs spatial recalibration by decomposing global context into separate horizontal and vertical directional components. First, global pooling is applied along the horizontal and vertical dimensions to create two direction-aware feature vectors (z_h and z_w). These vectors are then concatenated together into a single combined vector (z). This concatenated representation is processed through a Shared Multi-Layer Perceptron (SharedMLP) to learn the cross-channel and spatial relationships, outputting intermediate features a_h and a_w for the respective height and width dimensions. Directional attention maps are then generated by applying a sigmoid activation function (σ) to both a_h and a_w ,

and performing an element-wise multiplication ($\otimes$) between them to create the comprehensive attention map (A). Finally, the recalibrated output (y) is produced by multiplying the original input feature map (x) element-wise with this attention map (A), highlighting nematode-relevant spatial locations while suppressing homogeneous background noise.

Output format per detection: 4D bounding box, class logit, 32 mask coefficients enabling precise instance reconstruction. Inference reparameterization fuses DCA branches into equivalent 3×3 convolutions via kernel summation and BN folding, achieving VGG-like deployment speed (Ultralytics, 2025). This design, from pixel textures through curvature-aware anatomical feature extraction to prototype-based masks, precisely captures nematode morphology while optimizing edge deployment efficiency (Ding et al., 2021; Khanam & Ahmed, 2024). The architecture for the

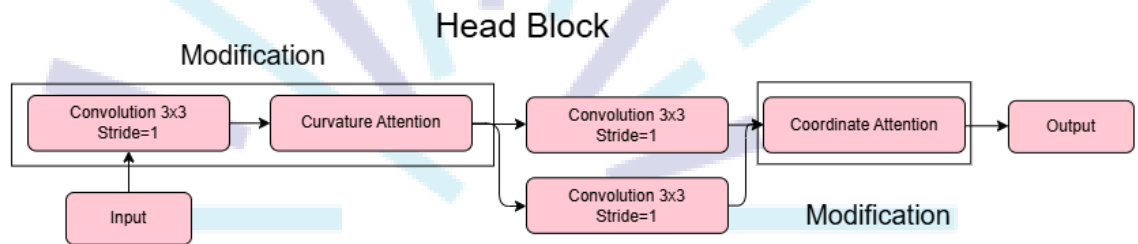


Figure 7 Head block for Yolov11n-seg-DCA model architecture

3.3.2. Classification Model: RepVGG-CA

RepVGG-CA is a modified Convolutional Neural Network (CNN) that trades off speed of inference with accurate localization of features. It is a lightweight architecture that is built by combining a custom, tinier architecture than the smallest RepVGG-A0 architecture and Coordinate Attention (CA) modules. The model is designed to be trained using a few resources of both the dataset and the training machine.

The model is founded on a custom, smaller version of RepVGG-A0, which is a VGG-like architecture suggested by Ding et al. The revised designation uses channel multipliers $a=0.5$ at the first four stages and $b=1.5$ at

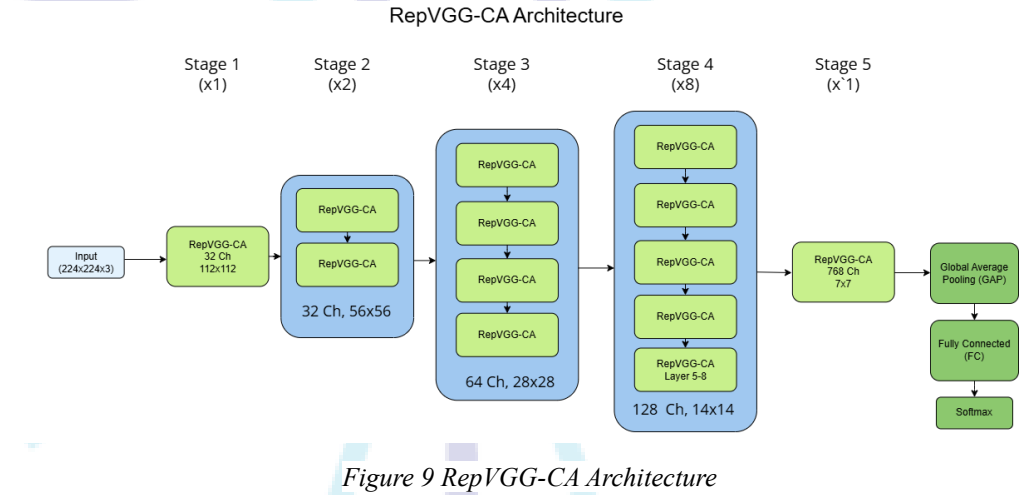
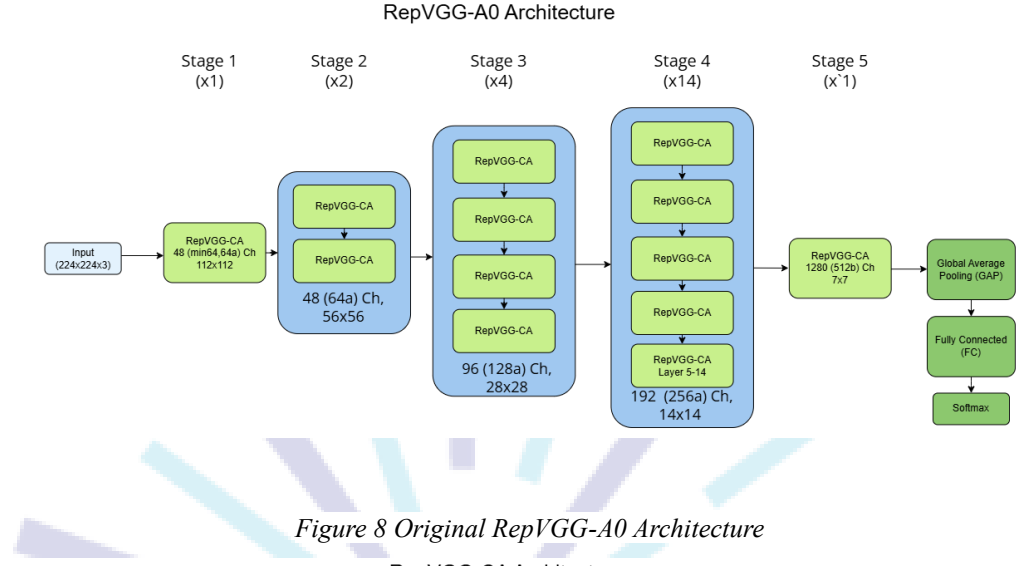
the last stage, resulting in a channel width per stage of 32, 32, 64, 128, and 768, respectively. The reduction from the standard $a=0.75$ and $b=2.5$ multipliers used in the RepVGG-A0 architecture, alongside the decrease in stage four depth from fourteen to eight blocks, is implemented to prevent overfitting.

The reduction from $a=0.75$ to $a=0.5$ halves the channel count at each of the first four stages, directly reducing the number of learnable parameters across the entire feature extraction backbone. At $a=0.75$, stages one through four produce [48, 48, 72, 144] channels; at $a=0.5$, these become [32, 32, 48, 96], cutting the parameter count at each convolutional layer by approximately 56% (since parameters scale with the square of channel width for same-stage convolutions). The value 0.5 is chosen over intermediate values such as 0.6 because it produces channel counts that are multiples of 8, maintaining alignment with GPU tensor core requirements without rounding. Similarly, b is reduced from 2.5 to 1.5, applying the same proportional reduction as a , bringing the final stage from 1280 to 768 channels and reducing the parameter count of the final convolutional block and the classifier input dimension. Stage four depth is reduced from fourteen to eight blocks, reducing the stage's depth by approximately 43% to lower overfitting risk at the highest-level semantic stage while retaining sufficient depth to capture complex morphological features such as body curvature and tail structure (Ding et al., 2021).

The generalized architecture of the original RepVGG-A0 is shown in Figure 5, while Figure 6 details the custom architecture of the paper's RepVGG-CA. Structural Re-parameterization is the characteristic of this base. The model employs a complicated multi-branch design (3x3 convolution, 1x1 convolution, and identity branch) during training, which enables a gradient flow and convergence. These branches, however, are fused mathematically together during the inference into a single 3x3 convolutional layer.

The RepVGG-CA classification backbone processes an input image. $X_0 \in \mathbb{R}^{3 \times 224 \times 224}$ Through a stem module followed by five sequential stages of RepVGG+CA blocks, each operating at fixed channel width and progressively

decreasing spatial resolution, terminating in a classification head that produces genus probabilities (Ding et al., 2021). The architecture follows a custom configuration with stage-wise channel dimensions of $C = [32, 32, 64, 128, 768]$ and block counts per stage $B = [1, 2, 4, 8, 1]$ for layers one to five, respectively, shown in Figure 6



Stage one receives the input and consists of exactly one RepVGG+CA block at 32 channels with a stride of 2, downscales the input to become 112×112 ; its single block output becomes $X_1 = X_{1,1} \in \mathbb{R}^{48 \times 112 \times 112}$. Stage two takes the output from stage one and applies two RepVGG+CA blocks at the same 32 channels: the first block downsamples via stride 2 to 56×56 , capturing short curves and local texture contrasts between nematode bodies and slide

background, while the second block (stride 1) refines these at fixed resolution, yielding $X_2 \in \mathbb{R}^{48 \times 56 \times 56}$. Stage three processes the output from stage two through 4 RepVGG+CA blocks that first expand to 64 channels while downsampling to 28×28 (Block 1, stride 2), then refine mid-level features like longer body segments and approximate poses across three additional stride-1 blocks ($X_3 \in \mathbb{R}^{96 \times 28 \times 28}$). Stage four, allocated the largest capacity with eight RepVGG+CA blocks at 128 channels, performs final downsampling to 14×14 at the first block and processes high-level morphological attributes such as overall body curvature, tail/head structure through seven refinement blocks, concentrating representational power where semantic detail dominates despite low resolution ($X_4 \in \mathbb{R}^{192 \times 14 \times 14}$). Stage five collapses remaining spatial structure via a single RepVGG+CA block at 768 channels with stride 2 downsampling to 7×7 , producing a compact global feature representation $X_5 \in \mathbb{R}^{1280 \times 7 \times 7}$ suitable for classification (Ding et al., 2021).

The classification head applies global average pooling (GAP) to each of the 768 channels of stage five. For each of the 768 channels C , the spatial mean across the $7 \times 7 = 49$ positions is computed. Variable $X_5(c, h, w)$ represents the activation at channel c , height h , and width w in stage five features. The $1/49$ normalization ensures each g_c has the same scale regardless of spatial extent. Result g encodes global channel statistics, removing all spatial information shown by equation 9 below.

$$g_c = \frac{1}{49} \sum_{h=1}^7 \sum_{w=1}^7 X_5(c, h, w), c = 1, \dots, 1280 \quad (9)$$

Followed directly the fully connected block (linear classifier) without intermediate hidden layers. Equation 10 below shows the fully connected module, where W is the learned weights, g_c is the output from global pooling, and b is for the biases learned.

$$z = W * g_c + b \in \mathbb{R}^C \text{ for } C \text{ number of classes,} \quad (10)$$

Producing final softmax probabilities for each class based on equation 11 below, where z_c is the logit for the class and z_j is the total logit across all

classes. The final predicted probability for a specific class C , denoted as p_c , is calculated by taking the exponential of the logit for that specific class, $\exp(z_c)$, and dividing it by the sum of the exponentials of all logits across all available classes, from index $j=1$ to C . This mathematical normalization ensures that the resulting output values fall strictly within a range of 0 to 1 and that the sum of the probabilities across all C classes exactly equals 1.0. This allows the network's final output to be interpreted as a well-calibrated confidence distribution over the possible nematode species.

$$p_c = \frac{\exp(z_c)}{\sum_{j=1}^C \exp(z_j)} \quad (11)$$

This stage-wise progression, from high-resolution shallow features to low-resolution deep semantics, precisely mirrors the hierarchical nature of visual recognition while allocating computational capacity optimally per the RepVGG design principles (Ding et al., 2021).

Each RepVGG+CA block within the stages is a multi-branch residual unit that, during training, employs parallel 3×3 convolution, 1×1 convolution, and (in non-downsampling blocks) identity branches whose outputs are summed prior to shared normalization, activation, and CA refinement; at inference, these branches are structurally re-parameterized into an equivalent single 3×3 convolution for VGG-like deployment efficiency through exact mathematical equivalence (Ding et al., 2021).

For a generic block receiving input $X_{k,b} \in \mathbb{R}^{C_k \times H_k \times W_k}$ where k is the stage (1-5), and $b-1$ refers to the input coming from the previous block. The diagram is shown in Figure 7 for training and Figure 8 for inference. The mathematical visualization is as follows in equation 12, where k is the current stage, where each branch includes its own batch normalization (BN) layer.

$$\begin{aligned} F_{3 \times 3} &= \text{BN}_3 \left(\text{Conv}_{3 \times 3}^{(C_k \rightarrow C_k, S_k, p=1)}(X_{k,b-1}) \right), \\ F_{1 \times 1} &= \text{BN}_1 \left(\text{Conv}_{1 \times 1}^{(C_k \rightarrow C_k, S_k)}(X_{k,b-1}) \right), \\ \text{Concat} &= F_{3 \times 3} + F_{1 \times 1}, \end{aligned} \quad (12)$$

$$U_{k,b} = \text{BN}_{\text{out}}(S_{k,b}),$$

$$F_{k,b} = \delta(U_{k,b}),$$

Equation 12 illustrates the multi-branch computational topology of a generic RepVGG block during the training phase, specifically for blocks that perform spatial downsampling. The input feature map from the previous block, denoted as $X_{k,b-1}$ (where k is the stage index and b is the block index), is simultaneously processed through parallel pathways. The first pathway applies a 3×3 convolution followed by Batch Normalization to produce the feature map $F_{3 \times 3}$. Concurrently, a second pathway applies a 1×1 convolution followed by its own Batch Normalization to generate $F_{1 \times 1}$. These parallel feature maps are then aggregated via element-wise addition to form the combined representation, *Concat*. This summed tensor is passed through an output Batch Normalization layer to create $U_{k,b}$, which is finally subjected to a Rectified Linear Unit (ReLU) activation function, yielding the post-activation output features ($F_{k,b}$).

Subsequent blocks within a stage (stride $s_k = 1$) add an identity shortcut branch with matching input and output dimensions, with the mathematical visualization as shown below in equation 13.

$$F_{\text{id}} = \text{BN}_{\text{id}}(X_{k,b-1}),$$

$$\text{Concat} = F_{3 \times 3} + F_{1 \times 1} + F_{\text{id}} \quad (13)$$

Equation 13 extends the multi-branch training topology for subsequent blocks within a stage where the input and output spatial dimensions remain identical (stride of 1). In addition to the 3×3 and 1×1 convolutional branches described previously, an identity shortcut branch is introduced to facilitate gradient flow and improve representation learning. This identity branch simply takes the input feature map ($X_{k,b-1}$) and passes it through a dedicated Batch Normalization layer to produce F_{id} . The aggregation step is therefore expanded so that the combined representation (*Concat*) is calculated as the element-wise sum of all three parallel pathways: the 3×3 convolutional branch $F_{3 \times 3}$, the 1×1

convolutional branch $F_{1 \times 1}$, and the identity branch F_{id} . Followed by the same shared output processing to produce pre-attention features $F_{k,b}$.

Structural Re-parameterization (Step-by-Step Mathematical Equivalence): The multi-branch training design equals a single 3×3 conv at inference via homogeneity (BN scales into convolution weights) and additivity (sum of convolutions is equal to convolution of summed kernels).

The first step is BN fusion: For input $M \in \mathbb{R}^{C_{in} \times H \times W}$, training output before the final ReLU is shown in Equation 14 below. For a given input tensor M, the total pre-activation output O is expressed as the summation of three distinct Batch Normalization operations applied to three separate branches. The first term $(BN^{(3)}(W^{(3)} * M))$ represents the 3×3 convolution branch with its weight matrix $W^{(3)}$. The second term $(BN^{(1)}(W^{(1)} * M))$ represents the 1×1 convolution branch with weight matrix $W^{(1)}$. The final term $(BN^{(0)}(M))$ represents the identity mapping branch where the input M is passed directly to the normalization layer. This equation sets the stage for mathematically fusing these discrete operations into a single equivalent convolutional layer for inference.

$$O = BN^{(3)}(W^{(3)} * M) + BN^{(1)}(W^{(1)} * M) + BN^{(0)}(M) \quad (14)$$

Each BN fuses into conv weights as in Equation 15 below, it details the specific mathematical transformation used to absorb the Batch Normalization parameters directly into the preceding convolutional weights and biases, a crucial step for inference optimization. The original Batch Normalization operation on a convolved input $(BN(W * M))$ is restructured into an equivalent, single linear operation $(W' * M + b')$. The new, fused convolutional weight matrix (W') is calculated by multiplying the original weights W by the Batch Normalization scale parameter (γ) and dividing by the square root of the running variance (σ^2) plus a small stability constant (ϵ). Similarly, the new fused bias term (b') is computed by taking the Batch Normalization shift parameter (β) and subtracting the product of the scale

parameter (γ) and the running mean (μ), divided by the same variance denominator.

$$\text{BN}(W * M) = W' * M + b' \text{ where } W' = W \cdot \frac{\gamma}{\sqrt{\sigma^2 + \epsilon}}, b' = \beta - \frac{\gamma\mu}{\sqrt{\sigma^2 + \epsilon}}. \quad (15)$$

The second is to pad $W'^{(1)}$ to 3×3 with zeros around the center, as can be seen in equation 16 below. Because the 1×1 convolutional branch possesses a smaller spatial footprint, its fused weight matrix, denoted as W'_1 , cannot be directly added to a 3×3 matrix. To resolve this, the 1×1 kernel is mathematically padded with zeros around its central value to expand its dimensions to match a 3×3 grid. The resulting zero-padded matrix is denoted as $\widehat{W}'_{1 \rightarrow 3}$. This zero-padding operation preserves the mathematical function of the original 1×1 convolution while ensuring structural compatibility, allowing it to be directly summed with the primary 3×3 convolutional kernel during the final reparameterization step.

$$\widehat{W}'_{1 \rightarrow 3} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & W'^{(1)} & 0 \\ 0 & 0 & 0 \end{bmatrix}. \quad (16)$$

The third step is to pad the identity BN-fused weights, padded with zeros around the center as follows in equation 17 below. Similar to the previous 1×1 padding step, the Batch Normalization-fused weights of the identity branch ($W'^{(0)}$) are padded with zeros around the center. This creates a structurally compatible 3×3 matrix, denoted as $\widehat{W}'_{0 \rightarrow 3}$, preparing it for direct integration with the other convolutional branches.

$$\widehat{W}'_{0 \rightarrow 3} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & W'^{(0)} & 0 \\ 0 & 0 & 0 \end{bmatrix}. \quad (17)$$

The fourth step is to sum all 3×3 kernels and biases as follows in equation 18 below. The final merged weight matrix (W_{final}) is the exact sum of the primary 3×3 weights ($W'^{(3)}$), the padded 1×1 weights ($\widehat{W}'_{1 \rightarrow 3}$), and the padded identity weights ($\widehat{W}'_{0 \rightarrow 3}$). The final bias (b_{final}) is similarly calculated by combining the fused biases from all three branches.

$$W_{\text{final}} = W'^{(3)} + \widehat{W}'_{1 \rightarrow 3} + \widehat{W}'_{0 \rightarrow 3}, b_{\text{final}} = b'^{(3)} + b'^{(1)} + b'^{(0)} \quad (18)$$

Then the fifth step is the final inference output feature map, the formula is as follows in equation 19 below. The final output feature map (O) is computed by applying a standard convolution using the aggregated kernel (W_{final}) on the input (M) and adding the combined bias (b_{final}).

$$O = W_{\text{final}} * M + b_{\text{final}}. \quad (19)$$

This proves the 3-branch + 4 BN topology compresses losslessly into one 3×3 conv, achieving $\sim 83\%$ faster inference than ResNet-50 equivalents (Ding et al., 2021). The block output $X_{k,b} = \text{CA}(F_{k,b})$ feeds the next block/stage.

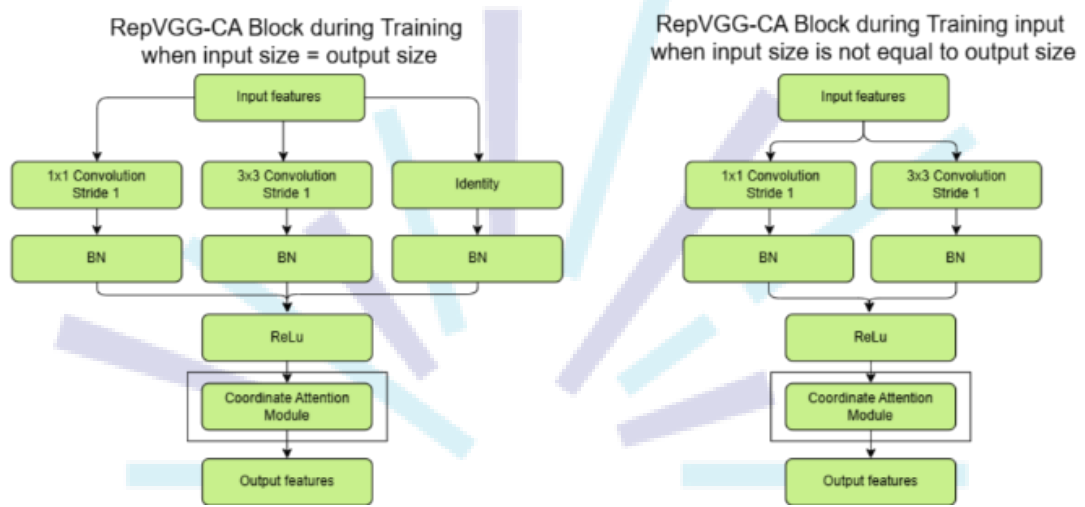


Figure 10 RepVGG block during training

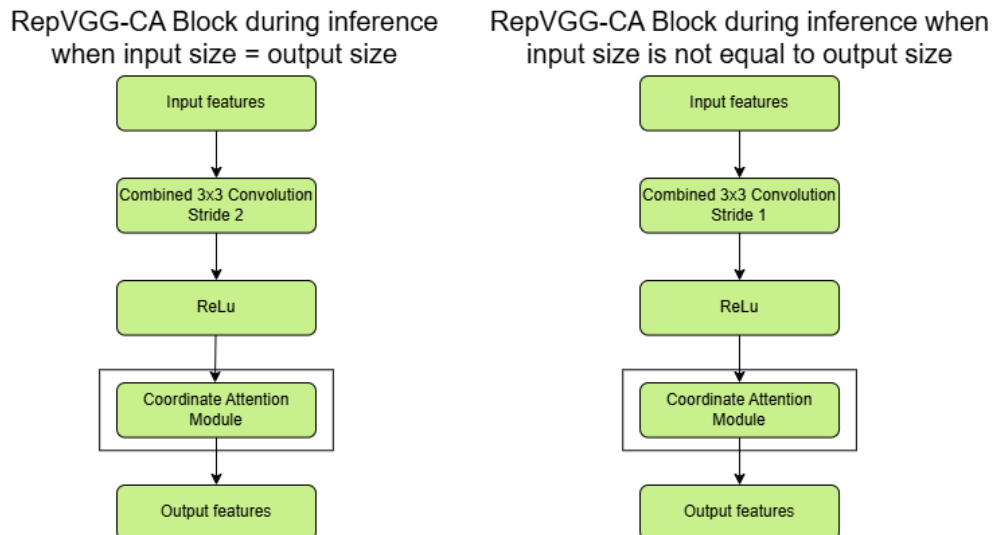


Figure 11 RepVGG block during inference

Although the simple RepVGG framework provides a very good base (shown in figure 10), with multi-branch training and 3x3 re-parameterization (shown in figure 11), it does not have spatial recalibration, which is essential to small microscopy images, where the model runs a risk of overfitting to homogeneous backgrounds of the slide, instead of the more discriminatory morphology of the nematode (Ding et al., 2021). Attention modules overcome this weakness by training adaptive weights to eliminate irrelevant activations (slide artefacts) and enhance task-relevant signals, e.g., body curvature and tail shape (Hou et al., 2021).

Coordinate Attention (CA) was specifically selected over channel-only Squeeze-and-Excitation (SE) or spatial attention because nematodes exhibit strong directional alignment (horizontal/vertical body length). SE's Global Average Pooling destroys spatial information necessary for long-range coordinate dependencies, whereas CA's coordinate-wise pooling explicitly preserves positional cues, enabling position-specific suppression of uniform microscopy backgrounds (Hou et al., 2021).

CA is applied post-ReLU at each block end to refine fused RepVGG features when spatial hierarchies are coherent, ensuring consistent refinement across stages without inference bottlenecks. Empirical validation confirms this synergy for classification: Hou et al. (2021) report +1-2% top-1 accuracy on ImageNet lightweight CNNs while biological imaging studies show attention mechanisms boost classification by focusing on anatomical structures against uniform backgrounds (Wang et al., 2023).

The Coordinate Attention module receives RepVGG features $F \in \mathbb{R}^{C \times H \times W}$ (e.g., C_k channels at stage resolution) and generates a position-sensitive channel attention map that emphasizes nematode-relevant spatial locations while suppressing homogeneous background regions through separate horizontal and vertical coordinate encoding (Hou et al., 2021). Unlike global average pooling that discards all spatial information, CA performs coordinate-wise pooling to preserve positional dependencies.

Equations 20 and 21 initiate the Coordinate Attention mechanism by performing directional pooling on the input feature map (F) to capture spatial statistics. Equation 20 extracts horizontal statistics, producing a height-aware feature map ($f_h(c, h)$) for a given channel (c) and height (h). This is achieved by taking the sum of the input features $F(c, h, w)$ across the width index (w) and dividing by the total feature map width (W) to calculate the average. Similarly, Equation 21 extracts vertical statistics to produce a width-aware feature map ($f_w(c, w)$) for a given channel (c) and width (w). This is calculated by averaging the input features ($F(c, h, w)$) across the height index (h), divided by the total feature map height (H).

$$f_h(c, h) = \frac{1}{W} \sum_{w=1}^W F(c, h, w) \in \mathbb{R}^{C \times H} \quad (20)$$

$$f_w(c, w) = \frac{1}{H} \sum_{h=1}^H F(c, h, w) \in \mathbb{R}^{C \times W} \quad (21)$$

capturing vertical (height-wise) and horizontal (width-wise) statistics, respectively. These are concatenated along the spatial dimension ($[f_h; f_w]$) and embedded through a shared bottleneck with a reduction ratio $r = 32$. Following the pooling, Equation 22 concatenates the vertical and horizontal statistical vectors, represented together as $[f_h; f_w]$, along the spatial dimension. This combined vector is then passed through a shared bottleneck utilizing a convolutional operation ($\text{Conv}_{7 \times 7}^{(2C \rightarrow C/r)}$) with a channel reduction ratio (r). A non-linear activation function (δ) is applied to the result. This generates a shared, compressed embedded representation (g) which effectively captures cross-channel relationships in a reduced dimensional space defined by C/r channels.

$$g = \delta(\text{Conv}_{7 \times 7}^{(2C \rightarrow C/r)}([f_h; f_w])) \in \mathbb{R}^{(C/r) \times (H+W)}, \quad (22)$$

Then g is split into height-aware $g_h \in \mathbb{R}^{((C/r) \times H)}$ and width-aware $g_w \in \mathbb{R}^{((C/r) \times W)}$ components. Separate 1D convolutions generate attention weights followed by sigmoid normalization. The shared representation (g) is

subsequently split back into its respective height-aware component (g_h) and width-aware component (g_w). Equations 23 and 24 process these split components to generate independent, position-sensitive attention weights. Equation 23 processes g_h through a 1D convolution that expands the channels back from C/r to the original C channels, followed by a sigmoid normalization function, sigma, yielding the horizontal attention weights ($a_h(c, h)$). Likewise, Equation 24 processes g_w through a similar 1D convolution and sigmoid normalization to produce the vertical attention weights ($a_w(c, w)$). Both sets of weights are constrained to a value range between 0 and 1.

$$a_h(c, h) = \sigma(\text{Conv}_{1 \times H}^{(C/r \rightarrow C)}(g_h)) \in (0,1)^{C \times H}, \quad (23)$$

$$a_w(c, w) = \sigma(\text{Conv}_{W \times 1}^{(C/r \rightarrow C)}(g_w)) \in (0,1)^{C \times W}, \quad (24)$$

Then the generated weights are broadcast-multiplied to form the full coordinate attention map with the equation 25 below. The final coordinate attention map ($A(c, h, w)$) is created for every channel (c), height (h), and width (w). This is achieved by performing a broadcasted element-wise multiplication between the horizontal attention weights ($a_h(c, h)$) and the vertical attention weights ($a_w(c, w)$).

$$A(c, h, w) = a_h(c, h) \cdot a_w(c, w) \in (0,1)^{C \times H \times W}. \quad (25)$$

The refined output is obtained via element-wise multiplication with equation 26 below. By performing an element-wise multiplication between the coordinate attention map $A(c, h, w)$ and the original features $F(c, h, w)$, the network produces the recalibrated output feature map ($Y(c, h, w)$). This output selectively highlights elongated nematode structures while effectively suppressing uniform background noise from the microscopy slide.

$$Y(c, h, w) = A(c, h, w) \cdot F(c, h, w) \in \mathbb{R}^{C \times H \times W}, \quad (26)$$

Effectively rescaling channels differently along height and width to highlight elongated nematode structures (typically aligned horizontally/vertically in microscopy fields) while down-weighting uniform slide background.

3.4. Training Strategy

Both YOLOv11n-seg-DCA and RepVGG-CA models are trained from random initialization on the augmented nematode dataset using a NVIDIA T4 GPU (16 VRAM).

3.4.1. YOLOv11-seg-DCA

Hyperparameters follow established YOLOv11 segmentation protocols (Wang et al., 2024; Shan et al., 2025). The input resolution is 320×320 pixels to balance maintaining details of the nematodes while optimizing computational throughput. Batch size per iteration is set to sixteen to leverage the gradient stability benefits that reduce gradient noise and improve convergence (Wang et al., 2024; He et al., 2025).

Stochastic Gradient Descent (SGD) with momentum is selected as the optimizer since it offers better generalization on small datasets compared to adaptive optimizers like Adam. SGD approximates the true gradient by averaging over random mini-batches, introducing stochastic noise that acts as an implicit regularizer against overfitting, especially critical on datasets with limited samples (Khanam & Ahmed, 2024; Wang et al., 2024). Momentum is set to 0.937, following Ultralytics' empirical findings to accelerate convergence by smoothing gradient updates while preventing oscillations (Khanam & Ahmed, 2024). Weight decay of 0.0005 is used for L2 regularization to constrain model complexity and further reduce overfitting risk (Khanam & Ahmed, 2024).

A cosine annealing learning rate scheduler is employed over the maximum epoch gradually decreasing the learning rate from an initial 0.0003 to near zero. The current learning rate (lr) is calculated at each epoch to ensure help nudge towards convergence. This is achieved by taking the initial learning rate (lr_0) and multiplying it by a cosine decay factor. This factor is computed by taking 1 plus the cosine of π multiplied by the ratio of the current training epoch to the maximum number of epoch. The entire result is then

divided by 2. This mathematical formulation gradually decreases the learning rate from its initial value down to near zero following a cosine curve, which acts as a regularizer to prevent overfitting, with the equation 27 as follows.

$$lr = lr_0 \cdot \frac{1 + \cos\left(\pi \cdot \frac{epoch}{max\ epoch}\right)}{2} \quad (27)$$

This smooth decay provides regularization by constraining model complexity and further reduces overfitting risk (Khanam & Ahmed, 2024).

The composite loss function follows Ultralytics YOLO11 defaults with a modified L_{dice} , as follows in equation 28.

$$L_{total} = 7.5L_{box} + 0.5L_{cls} + 1.5L_{dbox} + L_{seg} \quad (28)$$

(Ultralytics, 2025), where L_{box} (CIoU loss, weight=7.5) measures bounding box localization accuracy and receives the highest weight as precise detection fundamentally drives segmentation performance; L_{cls} (binary cross-entropy, weight=0.5) handles nematode classification with lower emphasis due to the single-class nature; L_{dbox} (distribution focal loss, weight=1.5) improves localization of challenging objects; and L_{seg} replaces the default L_{dice} , incorporating a curvature-weighted Dice loss prioritizing nematode boundaries, with the equation for the L_{seg} shown in equation 29 below

$$L_{seg} = L_{dice} \cdot (1 + |\nabla^2 M|) \quad (29)$$

Standard Dice loss may inadequately supervise nematode boundaries due to uniform pixel weighting that under-penalizes errors in sparse high-curvature regions (heads, tails, bends comprising <15% mask area) (Kervadec et al., 2021). This curvature-weighted variant upweights biologically critical regions where Laplacian $|\nabla^2 M|$ peaks. The curvature term $\nabla^2 M$ (Laplacian of ground-truth masks) highlights high-curvature boundary pixels characteristic of nematode undulations (Shan et al., 2025; Ultralytics, 2025). The formula for the laplacian is in equation 30 as follows

$$\nabla^2 M = \partial^2 M / \partial x^2 + \partial^2 M / \partial y^2 \quad (30)$$

The Laplacian operator, a second-order derivative filter, measures areas of rapid intensity change by calculating the sum of second derivatives in

horizontal and vertical directions (OpenCV, 2025; Roboflow, 2025). $\nabla^2 M$ utilized in the curvature-weighted segmentation loss. As a second-order derivative filter applied to the mask M , it measures areas of rapid intensity change. It is calculated as the sum of two components: the second partial derivative of the mask with respect to the horizontal spatial dimension ($\partial^2 M / \partial x^2$), and the second partial derivative with respect to the vertical spatial dimension ($\partial^2 M / \partial y^2$).

Sharp corners, such as head or tail transitions in nematodes, produce stronger Laplacian responses while smoother bends yield lower values. Straight segments have near-zero Laplacian response because the intensity gradient changes little. These values depend on the discrete convolution with standard Laplacian kernels (e.g., 3×3 kernel with center -4 and neighbors 1) applied on the binary ground-truth masks, and thus vary slightly depending on mask resolution and kernel design (OpenCV, 2025; Tutorials Point, 2024). They serve to guide the weighting scheme in the loss function, emphasizing biologically critical features by amplifying errors where Laplacian magnitudes peak.

Below in table 6 is the parameter summary for the Yolov11n-seg-DCA

Table 6 Parameter Summary for Yolov11n-seg-DCA

Parameter	Value
Input Resolution	320 x 320 pixels
Training Epochs	200
Batch Size	16
Optimizer	Stochastic Gradient Descent (SGD)
Momentum	0.937
Weight Decay (L2 Regularization)	0.0005
Initial Learning Rate	0.0003
Configuration	Value

3.4.2. RepVGG-CA

RepVGG CA is trained as an image classifier using batches of 16 images, which fits well within the memory of the NVIDIA Tesla T4 while still providing stable gradient estimates for the small dataset. The optimizer is AdamW, because its adaptive per-parameter learning rates and decoupled weight decay generally give faster and more stable convergence than classical SGD when training deep networks from scratch on limited data (Loshchilov & Hutter, 2019). A weight decay factor of 0.005 is applied to the coordinate attention and linear layer to regularize the model by penalizing very large parameter values. Main branches receive a separate L2 penalty following the original RepVGG paper, while biases and batch normalization parameters use a weight decay of 0.0 (Ding et al., 2021). This encourages smoother, lower norm solutions and reduces overfitting to the small training set, which is particularly important when the model capacity is high relative to the number of training images (Loshchilov & Hutter, 2019).

The learning rate schedule combines linear warmup and cosine decay. During the first 15 epochs, the learning rate is increased linearly from 0 to 5×10^{-4} . Warmup avoids unstable, overly large updates at the start of training, when the network weights and batch normalization statistics are still far from a sensible operating regime. After epoch 15, the learning rate follows a cosine decay schedule over the remaining epochs so that large steps are taken early, and progressively smaller refinements are made near convergence, which has been shown to improve optimization behaviour in deep networks without requiring manual step decay tuning (Loshchilov & Hutter, 2017).

Learning rate after the fifteenth epoch is as shown in equation 31 below. Following an initial linear warmup period, the learning rate at a specific epoch e ($lr(e)$) is gradually reduced. It is calculated by taking the base learning rate (lr_0) and multiplying it by a cosine decay factor. This decay factor is 1 plus the cosine of pi multiplied by the ratio of the current epoch minus the warmup threshold ($epoch - 15$) to the adjusted maximum epochs ($max\ epoch - 15$),

with the entire term divided by 2. This schedule allows for large weight updates early in training and progressively smaller refinements as the model approaches larger epochs.

$$lr(e) = lr_0 \cdot \frac{1 + \cos \left(\pi \cdot \frac{epoch - 15}{\max epoch - 15} \right)}{2} \quad (31)$$

The classification loss is the standard multi-class cross-entropy, modified with label smoothing $\epsilon=0.1$ to improve generalization and calibration. Let p ($p_1, \dots, p_k$) be the softmax output over 2 (K) nematode species, and let y be the ground truth class. Plain cross entropy uses hard one-hot targets (exactly 1.0 for true class, 0.0 for others), which encourages overconfident predictions that overfit small datasets. Label smoothing replaces this with softened targets $1-\epsilon$ for the true class and $\epsilon/(K-1)$ for others, which acts as regularization by preventing extreme logit values and has been shown to consistently improve test accuracy by 0.5-2% across vision tasks while better aligning predicted confidence with true accuracy (Szegedy et al., 2016; Müller et al., 2019). The smoothed cross-entropy loss formula is shown in equation 32 below

$$L_{CE,smooth} = - \sum_{k=1}^K t_k \log p_k \quad (32)$$

Where t_k are the smoothed target probabilities. This alteration is particularly valuable for the nematode dataset, where microscopy images may contain label noise (blurry focus, ambiguous worms) and the augmented training set remains relatively small; class frequencies are only moderately imbalanced, so no additional focal weighting is needed.

Finally, structural reparameterization is applied after training to make the network efficient at inference. During training, RepVGG blocks use a multi-branch structure (3×3 , 1×1 , identity) for better representation learning, but this is computationally expensive at inference; reparameterization analytically fuses these branches into a single 3×3 convolution per block, preserving the exact function while reducing operations by $\sim 2 \times$ (Ding et al., 2021).

Below in table 7 is the parameter summary for the RepVGG-CA:

Table 7 HyperParameter Summary for RepVGG-CA

Configuration	Value
Input Resolution	224 x 224 pixels
Training Epochs	80
Batch Size	16
Optimizer	AdamW
Weight Decay	0.005
Base Learning Rate	0.0005
Warmup Epoch	15
Learning Rate Scheduler	Cosine decay after warmup
Label Smoothing	0.1

3.5. Evaluation Metrics

Task-specific metrics, which refer to the goals of the two-stage pipeline, i.e., accurate cases of nematodes and followed by classification of the species, are used to measure the performance of the model. The YOLOv11n-seg-DCA detection/segmentation network is evaluated through the mean Average Precision (mAP) in an intersection-over-union threshold of 0.5 (mAP50) for both the box and segmentation mask. Equation 33 below defines the Mean Average Precision evaluated at an extremely strict Intersection over Union threshold of 0.5 (mAP50). The mAP50 formula where C is the number of classes and $AP_c(t)$ is the average precision for class C at IoU threshold 0.5 is shown in the equation 33 below for both the mask and the box. The metric is calculated by taking the average precision (AP_c) for a specific class c evaluated at the $\text{IoU} \geq 0.5$ condition. These individual class precisions are summed across all classes from $c=1$ to the total number of classes C, and the final result is divided by C to yield the mean score.

$$\text{mAP50-mask} = \frac{1}{C} \sum_{c=1}^C AP_c^{\text{mask}}(\text{IoU} \geq 0.5,) \quad (33)$$

$$\text{mAP50-box} = \frac{1}{C} \sum_{c=1}^C \text{AP}_c^{\text{box}} (\text{IoU} \geq 0.5)$$

The mAP50-box metric measures bounding-box detection accuracy, while the mAP50-mask metric evaluates instance segmentation accuracy in alignment with the COCO instance-segmentation standard procedure (Ultralytics, 2025). These measures give a quantitative estimate of the model to localise nematodes within textured microscopy backgrounds and define their curvilinear contours in an accurate way. The mAP50-95 metric is a balance between detection precision and recall over confidence levels, but mAP50-95 is a balance between pixel-level overlap between predicted mask and ground-truth annotations, which is necessary in downstream region-of-interest (RoI) extraction. The metric averages the precision over ten distinct Intersection over Union thresholds (t). These thresholds range from 0.50 to 0.95 in increments of 0.05. For each class c up to the total classes C , the average precision $\text{AP}_c(t)$ is calculated at every threshold, summed together, and divided by 10. The results for all classes are then averaged together by dividing by C .

$$\begin{aligned} \text{mAP50-95-box} &= \frac{1}{C} \sum_{c=1}^C \frac{1}{10} \sum_{t \in \{0.5, 0.55, \dots, 0.95\}} \text{AP}_c^{\text{box}}(t) \\ \text{mAP50-95-mask} &= \frac{1}{C} \sum_{c=1}^C \frac{1}{10} \sum_{t \in \{0.5, 0.55, \dots, 0.95\}} \text{AP}_c^{\text{mask}}(t) \end{aligned} \quad (34)$$

Equation 35 below defines the Mean Average Precision evaluated at an extremely strict Intersection over Union threshold of 0.8 (mAP80). The metric is calculated the same way as equation 33 with a higher threshold. The mAP80 metric would test the precise localization capabilities of the model. While mAP50 confirms basic detection, its 50 percent overlap tolerance permits imprecise bounding boxes and masks. The RepVGG-CA classification model relies on morphological cues, such as tail shape, to differentiate between nematode species. By enforcing the stricter 80 percent overlap, mAP80 measures the model's ability to achieve a tighter pixel alignment.

$$\begin{aligned} \text{mAP80-mask} &= \frac{1}{C} \sum_{c=1}^C \text{AP}_c^{\text{mask}} (\text{IoU} \geq 0.80,) \\ \text{mAP80-box} &= \frac{1}{C} \sum_{c=1}^C \text{AP}_c^{\text{box}} (\text{IoU} \geq 0.80) \end{aligned} \quad (35)$$

In addition to these metrics, Precision and Recall are utilized. Precision for a specific class c (P_c , shown in equation 36), is calculated by taking the total number of True Positives (TP_c , correctly predicted instances of that class) and dividing it by the sum of True Positives and False Positives (FP_c , instances incorrectly predicted as belonging to that class). This metric quantifies the model's ability to avoid hallucinating false detections or misclassifications.

$$\text{Precision}_c(P_c) = \frac{TP_c}{TP_c + FP_c} \quad (36)$$

Recall (shown in equation 37) measures the system's ability to successfully identify all actual positive instances. Recall for a specific class c (R_c) is calculated by dividing the number of True Positives (TP_c) by the sum of True Positives and False Negatives (FN_c , actual instances of the class that the model completely missed).

$$\text{Recall}_c(R_c) = \frac{TP_c}{TP_c + FN_c} \quad (37)$$

The classification phase represented by the RepVGG -CA architecture is assessed using traditional multi-class performance measures, such as Precision (shown in equation 36 above), Recall (shown in equation 37 above), Accuracy, and F1-Score. Equation 38 below shows the overall Accuracy, evaluating the model's general classification performance across all nematode species. It divides the total correct predictions by the total evaluated instances using aggregated confusion matrix components. The numerator combines overall True Positives (TP_{overall} , instances correctly predicted as the target species), and overall True Negatives (TN_{overall} , instances correctly identified as non-targets). This sum is divided by the denominator, which represents all evaluated instances: TP_{overall} , TN_{overall} , overall False Positives (FP_{overall} , incorrect target predictions), and overall False Negatives (FN_{overall} , actual targets the model missed).

$$Accuracy_{overall} = \frac{TP_{overall} + TN_{overall}}{TP_{overall} + TN_{overall} + FP_{overall} + FN_{overall}} \quad (38)$$

Equation 39 below defines the F1-Score ($F1_c$) for a specific class c . This metric is the harmonic mean of the Precision (P_c) and Recall (R_c) for that class. It is calculated by multiplying 2 by the Precision and Recall, then dividing that product by the sum of the Precision and Recall.

$$F1_c = 2 \cdot \frac{P_c \cdot R_c}{P_c + R_c} \quad (39)$$

These are measures that determine how well the model discriminates between the two species, *Meloidogyne incognita* and the species, *Meloidogyne graminicola* based on refined morphological cues obtained on rigorously mask-cropped regions of interest (RoIs). A confusion matrix is used to plot the patterns of misclassification, which is most useful in identifying subtle anatomical differences, including tail morphology and body curvature ratio. Equation 40 below represents the structure of a Confusion Matrix. The matrix is composed of four quadrants: True Positives (TP) and True Negatives (TN) on the diagonal representing correct predictions, alongside False Positives (FP) and False Negatives (FN) representing specific types of errors.

$$Confusion\ matrix = \begin{bmatrix} TP & FN \\ FP & TN \end{bmatrix} \quad (40)$$

System-level evaluation combines the two steps through an end-to-end mAP calculated using full-field microscopy images, thus estimating the detection-to-classification error without cropping in between. Computational viability Inference latency (frames per second), and parameter count (millions of parameters) can be used as a measure of how the model can run on resource-constrained hardware. By comparing the baseline YOLOv11n-seg model with a curvature-weighted one, ablation experiments review the effect of the curvature-aware attention (DCA) module; these comparisons demonstrate hundreds of boundary IoU improvements and corresponding concomitant mAP gains.

The performance measures are based on a held-out set of 10 % of the data as a result of the 70% training, 20% validation, and 10% testing split which ensures

an unbiased evaluation of the generalization ability. The obtained overall evaluation model justifies the efforts of every model element and establishes the effectiveness of the integrated pipeline in the identification of nematodes within the microscopy processes.

Model performance will be comprehensively evaluated using both theoretical computational complexity (FLOPs) and practical inference speed (FPS) metrics across the detection model (YOLOv11n-seg-DCA), classification model (RepVGG-CA), and full end-to-end pipeline. Equation 41 below calculates the theoretical computational complexity of the convolutional neural network, measured in Floating Point Operations, or FLOPs. For a given convolutional layer, it is calculated by multiplying 2 by the number of input channels (C_{in}), the number of output channels (C_{out}), the kernel height (K_h), the kernel width (K_w), the output feature map height (H_{out}), and the output feature map width (W_{out}). These values are summed across all layers to determine the model's total computational burden.

$$\text{FLOPs} = 2 \times C_{in} \times C_{out} \times K_h \times K_w \times H_{out} \times W_{out} \quad (41)$$

Equation 43 below defines the practical throughput of the model, measured in Frames Per Second (FPS). It is calculated as the inverse of the inference latency, taking 1 divided by the total time in seconds required to fully process a single image ($t_{inference}$).

$$\text{FPS} = \frac{1}{t_{inference}} \quad (42)$$

Ablation studies will be incorporated to validate the chosen training configurations, specifically including an optimizer ablation for the YOLOv11n-seg-DCA model to evaluate various Stochastic Gradient Descent strategies and a hyperparameter tuning assessment using Optuna for the RepVGG-CA classification model. Furthermore, the suggested architectures for the detection and classification models will be benchmarked with the same metrics mentioned for each respective model type, with the unmodified baseline models and the models previously used by existing literature. Namely, YOLOv11n-seg-DCA will be compared to YOLOv11n-seg without the Deformable Curvature Attention (DCA) module, and

RepVGG-CA will be compared to the usual RepVGG-A0 without Coordinate Attention (CA). These comparisons will be able to isolate the contribution of the DCA and CA modules. Moreover, the new models will be evaluated in comparison to YOLOv5-608, which is used by Pun et al. (2023) to detect nematodes, and to EfficientNetV2B0, which is used by Shabrina et al. (2023) to perform classification, which will guarantee a comparison with the previous models that have achieved good results.



CHAPTER IV

RESULT AND DISCUSSION

4.1. Object Detection Model (Yolov11-seg-DCA)

4.1.1. Training

The YOLOv11n-seg-DCA model was trained using a Kaggle computational environment with two NVIDIA Tesla T4 GPUs.

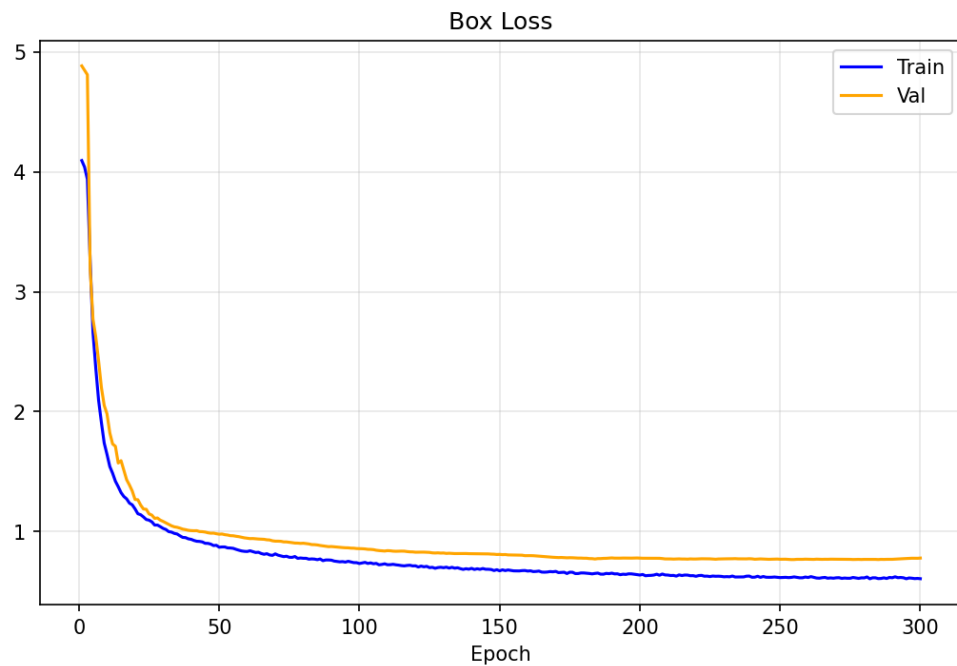


Figure 12 Yolov11-seg-DCA Train Box-loss

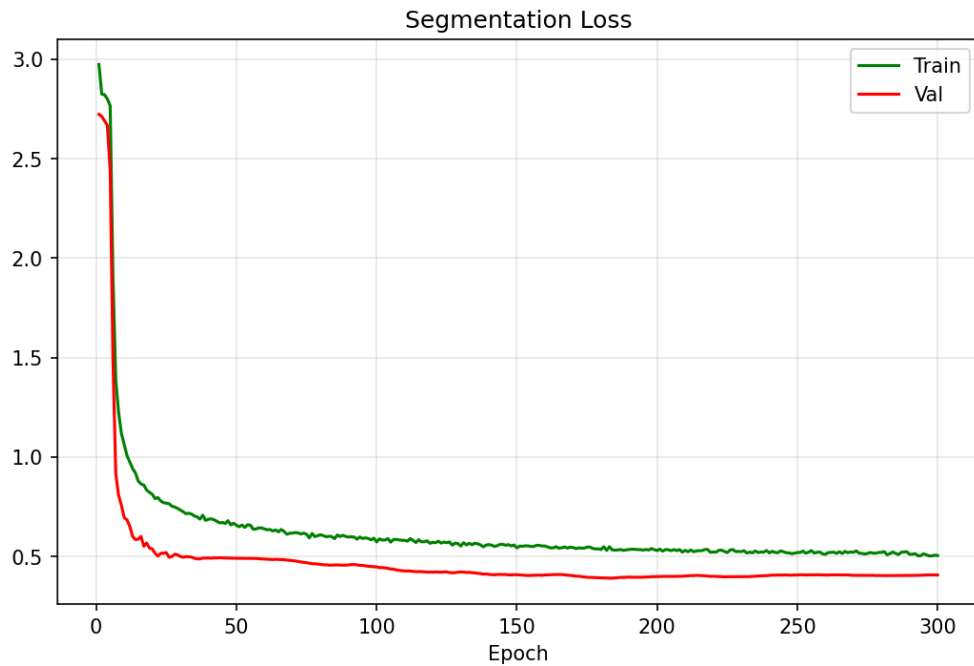


Figure 13 YOLOv11-seg-DCA Train Segmentation-loss

The bounding box regression loss and segmentation mask loss of the YOLOv11-seg-DCA model are shown in Figure 12 and Figure 13, respectively, during 300 epochs of training. Both loss functions show very rapid and efficient convergence during the early stage of training (epoch 0, approximately epoch 25) with a rapid drop in error. The bounding box loss on the train set drops from an initial value around 4.1 to about ~ 0.6 , and the validation loss experiences a parallel drop from nearly 4.9 down to ~ 0.4 . At the same time, the training segmentation loss decreases from ~ 3.0 to ~ 0.5 , and the validation segmentation loss decreases significantly from 2.7 to ~ 0.4 . This coordinated fast descent elucidates an effective initialization strategy and an optimal learning rate schedule, through which the model can learn the properties and structural layout of nematode instances in the dataset.

After a steep drop, both losses go into an optimization phase that is gradual and smooth. The training curves gradually decrease between epochs 50 and 150, indicating that the network is continually improving its localization and segmentation over time. A unique feature noticed about the segmentation loss is that the validation curve is always below the training curve throughout

the entire 300-epoch run. This means that the heavy regularizations are able to reduce the tendency to memorize the training data and to result in more generalizable feature maps.

After epoch 150, both evaluation metrics are able to form very consistent plateaus until epoch 300, when the training is finished. The validation bounding box loss stays relatively stable around 0.75 to 0.78, and the training bounding box loss ends at about 0.6. At the same time, the validation segmentation loss stays consistent at around 0.4, and the training segmentation loss is getting closer to 0.5. The training and validation curves stay very close and level over the last 150 epochs for both of the metrics. Most importantly, neither validation curve shows any up-shift, volatility, or late-stage jitter. The absence of an upward trend for all the validation metrics confirms that the YOLOv11-seg-DCA model is not overfitting, thus it shows good generalization ability, and the box localization and boundary mask prediction are also reliable.

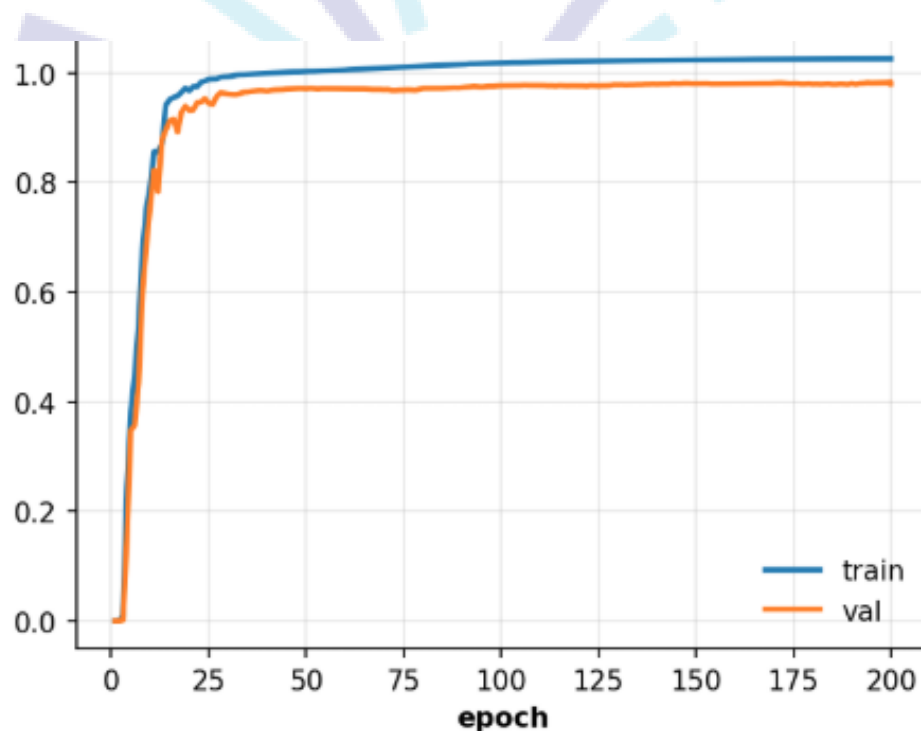


Figure 14 YOLOv11-seg-DCA mAP50 (box)

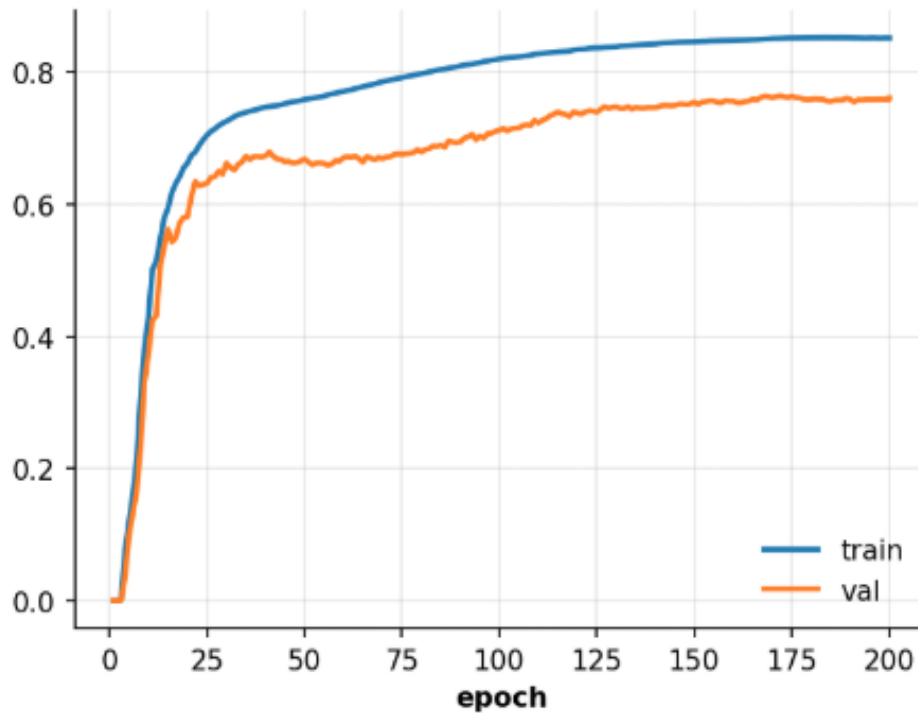


Figure 15 Yolov11-seg-DCA mAP50-95 (box)

For target identification capability, Figure 14 displays the mAP at an Intersection over Union (IoU) threshold of 0.50 (mAP50). The more stringent mAP is calculated across a range of IoUs from 0.50 to 0.95 (mAP50-95) in Figure 15. The accuracy metrics demonstrate this with a tremendous jump in accuracy during the initial 25 epochs, as the bounding boxes and segmentation loss curves also show quick convergence. The mAP50 curve starts from 0.0 and goes straight up to cross the 0.90 threshold by epoch 20, showing that the network quickly identifies the distinctive spatial characteristics of the nematodes. Similarly, the mAP50-95 curve in this early stage rapidly increases to ~0.70 at epoch 25, showing how the learning rate schedule and network initialization enable YOLOv11-seg-DCA to quickly switch from a random distribution of weights to a highly effective localization pattern.

These metrics continue to improve and stabilize in the later stages of training, solidly showing no over-optimization. The mAP50 curve continues to build on its success and sustains a high level of mAP above 0.95 until it slowly climbs to near 0.99 from epoch 25 through 200, whereas the more difficult

mAP50-95 curve gradually increases, going from 0.70 to approximately 0.81. This continuous improvement is using a higher-level scoring scheme, which is a testament to the dynamic updating of BB edges at a per-pixel level with the network's learning during epochs after training, which directly benefits from the training epochs avoiding saturation due to the combinatorial data augmentation. Both show very high and stable values in the final 100 epochs, with mAP50 in the range of 0.99 and mAP50-95 in the range of 0.81-0.82. The absence of any general downtrend or volatility in the data at any particular point, or any sudden drop in performance, is the best proof of the model's ability to generalize well to untouched data, and its YOLOv11-seg-DCA model has left us with excellent localization accuracy and well-optimized loss curves.

The Deformable Curvature Attention's capability to capture complex features comes through this, which illustrates the effectiveness of the module. Convolutional layers traditionally do not work well for elongated objects due to their fixed grid receptive fields combining adjacent features. The Deformable Curvature Attention module, on the other hand, dynamically moved the positions of the samples based on the winding body of the nematodes, which would make the sampling points strictly aligned to the nematode bodies. This confirms that the architecture is able to define specimens in complex microscopy backgrounds. The high precision sustained, particularly, is proof that the model can learn to differentiate fine textures in foreground and similar textures in the slides, without giving false positives, indicating the highly robust performance of the model in the part of the feature extraction layer.

4.1.2. Result and Comparison

The 10% test dataset was utilized to assess the performance of the YOLOv11n-Seg-DCA architecture. The primary metrics employed for this evaluation include Precision, Recall, and mean Average Precision (mAP) at an Intersection over Union (IoU) threshold of 0.50 (mAP50), as well as across a

gradient of thresholds from 0.50 to 0.95 (mAP50-95) for both bounding box localization and segmentation masks.

To justify the selection of the optimizer for the YOLOv11n-seg-DCA model, an ablation study was conducted. The study evaluated the performance of three variations of the Stochastic Gradient Descent (SGD) optimizer. The configurations tested were SGD with Momentum (the proposed approach), SGD Batch, and SGD Mini-Batch without momentum. The evaluation was performed on the 10% test dataset to observe how each optimizer handles the morphological details of the nematode samples. The performance metrics for bounding box localization and segmentation masks are presented in table 8 for box result and table 9 for the segmentation results.

Table 8 Bounding Box Performance Comparison for Optimizer

Model Architecture	Precision	Recall	mAP50	mAP50-95	mAP80
SGD with Momentum (Proposed)	96.67%	100.00%	99.30%	84.87%	92.89%
SGD Batch	0.12%	13.79%	0.07%	0.02%	0.00%
SGD Mini-Batch	55.00%	75.86%	64.23%	20.89%	6.62%

Table 9 Segmentation Performance Comparison for Optimizer

Model Architecture	Precision	Recall	mAP50	mAP50-95	mAP80
SGD with Momentum (Proposed)	96.67%	100.00%	99.30%	70.27%	72.45%
SGD Batch	0.00%	0.00%	0.00%	0.00%	0.00%
SGD Mini-Batch	60.53%	79.31%	74.86%	34.80%	4.04%

The ablation study results demonstrate a stark contrast in model convergence and detection capabilities depending on the optimizer configuration. The proposed SGD with Momentum achieved the highest performance across all metrics, recording a bounding box mAP50 of 99.30% and a segmentation mask mAP50 of 99.30%. The inclusion of momentum allowed the model to navigate the complex loss landscape of the small microscopy dataset by accumulating gradients over consecutive steps. This mechanism accelerates convergence and prevents the model from getting trapped in local minima because it introduces inertia to the weight updates. When gradients point in a consistent direction over multiple iterations, momentum amplifies the step size in that direction. Conversely, when noisy gradients from complex backgrounds cause oscillating directions, momentum dampens those oscillations. This smoothing effect is critical for extracting fine morphological details without getting derailed by background artifacts.

In contrast, the SGD Mini-Batch approach without momentum yielded significantly degraded results, achieving only 64.23% bounding box mAP50 and 74.86% mask mAP50. Without the velocity provided by momentum, the model struggled to generalize the features of the nematodes and suffered from noisy gradient updates. This degradation occurs because the gradient calculated from a small mini-batch is only a rough estimate of the true dataset gradient. Without momentum to average out this noise, the model takes erratic steps in the parameter space, often oscillating across the slopes of the loss function rather than descending smoothly toward the optimal minimum. Furthermore, the SGD Batch approach failed entirely to converge, resulting in near-zero precision, recall, and mAP scores for both bounding boxes and segmentation masks. Processing the entire dataset in a single batch smoothed out the stochastic noise necessary to escape saddle points, causing the model training to stall completely. Because a full-batch gradient calculates the exact direction of steepest descent for the entire dataset at once, the updates are entirely

deterministic. If this deterministic path leads into a flat region or a saddle point in the loss landscape, the model lacks the random variance required to bump the weights out of that trap. These findings conclusively validate the selection of SGD with Momentum as the optimal configuration for training the YOLOv11n-seg-DCA architecture in this specific domain.

An analysis of a number of well-known baseline models was performed to validate the proposed architectural improvements for the object detection model. In this benchmark, The chosen the architectures are the following: YOLOv5-608 architecture from Pun et al. (2023) with the parameters as stated in the above paper. Along with the baseline YOLOv11n-seg model with the same parameters as the proposed YOLOv11n-Seg-DCA.

Table 10 Bounding Box Performance Comparison

Model Architecture	Precision	Recall	mAP50	mAP50-95	mAP80
YOLOv5	96.67%	100.00%	98.00%	84.60%	98.51%
YOLOv11n-seg (base)	96.67%	100.00%	98.93%	85.70%	87.94%
YOLOv11n-Seg-DCA (Proposed)	96.67%	100.00%	99.30%	85.36%	92.89%

Table 11 Segmentation Performance Comparison

Model Architecture	Precision	Recall	mAP50	mAP50-95	mAP80
YOLOv5	N/A	N/A	N/A	N/A	N/A
YOLOv11n-seg (base)	96.67%	100.00%	98.93%	71.71%	70.83%
YOLOv11n-Seg-DCA (Proposed)	96.67%	100.00%	99.30%	69.51%	72.45%

The segmentation performance of YOLOv5 in Table 11 is not evaluated (N/A) because the original YOLOv5 architecture is designed solely for box object detection and does not provide instance segmentation capabilities. Consequently, no segmentation metrics can be reported for YOLOv5, as pixel-level mask predictions are not generated by the model. When the proposed model was tested with the data test, its performance in detecting and segmenting the data was found to be very good. The more challenging bounding box mAP50-95 came in at 85.36%, the harder mAP50 at 99.30%, and the strict mAP80 metric achieved 92.89%. Likewise, the metrics of instance segmentation had an mAP50 of 99.30%, an mAP50-95 of 69.51%, and a mAP80 of 72.45%. Higher scores reflect good ability to find, frame, and discriminate nematode structures in complex visual environments.

The successful outcome was directly enabled by the Deformable Curvature Attention (DCA) module, which captured the curvilinear edges of *Meloidogyne* species with precision at the pixel level. In addition, in both the bounding box and mask evaluations, the model obtained a Recall of 100.00%, which means that there were no target nematodes missed. Further, the test precision for both the bounding boxes and segmentation masks were 96.67%. This is consistent with the architecture that reduces the risk of hallucinating nonexistent objects. Creating the bounding boxes and pixel-precise masks without a minimum number of false discoveries.

It is also important to notice the transition in localization priorities in the comparative review of the bounding box metrics in Table 10. The baseline of the legacy YOLOv5 was decent with an mAP50 of 98.00%, an mAP50-95 of 84.60%, and a high mAP80 of 98.51%. This was improved by the base YOLOv11n-seg model, which had an mAP50 of 98.93% and an mAP50-95 of 85.70%, although its mAP80 decreased to 87.94%. The proposed YOLOv11n-Seg-DCA architecture obtained the best absolute localization accuracy at the primary threshold, with a mAP50 score of 99.30%, which dropped slightly to 85.36% for mAP50-95, but maintained a strong mAP80 of 92.89%.

This operational trade-off is further highlighted by the instance segmentation metrics that are reported in Table 11. The YOLOv11n-Seg-DCA model outperformed the YOLOv11n-seg baseline model in terms of segmentation mAP50, with 99.30% compared to the baseline model of 98.93%, as well as in the mAP80 metric, scoring 72.45% against the baseline's 70.83%. The baseline model had a higher Multi-threshold mask metric of 71.71% than the proposed model, with a metric of 69.51%.

The overall performance drop of this metric on both the bounding box and segmentation mask levels is acceptable and desired based on the architecture targets of the pipeline. The mAP50-95 is a strict metric that requires adherence to the boundaries of all objects with a 50% to 95% overlap. The major purpose of this particular stage of the pipeline is to detect a general location of the nematode and to propose a bounding box that can be used to crop a nematode from the image for the classification stage to come. Just a centered and well-framed Region of Interest (RoI) is required for the classification model RepVGG-CA, and does not need perfect contour tracing. Thus, the optimal mAP50 to 99.30% guarantees that the maximum number of valid nematodes will be identified and sent to the classifier. The Deformable Curvature Attention module has been able to optimize for this fundamental detection confidence over hyper-precise edge localization.

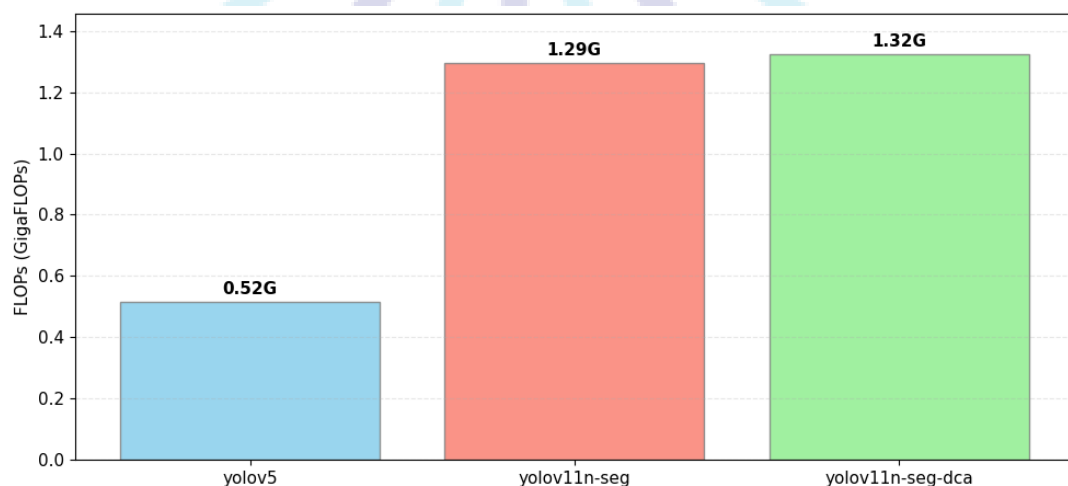


Figure 16 Object detection model complexity

Observing Figure 16 above, the legacy YOLOv5 architecture is the lightest of the group and needs just 0.52 GFLOPs. However, the drawback of this relatively low computational cost is that it cannot be used to extract precise morphological information due to the absence of instance segmentation. YOLOv11n-seg performs bounding box localization and pixel-level segmentation with a baseline model of 1.29 GFLOPs. The proposed YOLOv11n-seg-dca architecture runs at 1.32 GFLOPs. This means that the Deformable Curvature Attention (DCA) module adds a negligible 0.03 GFLOPs or less than 2.5% to the baseline. This small rise exemplifies a very efficient architectural solution.

The previous paper's YOLOv5 had the lowest model complexity due to it being merely being able to box the object detected rather than masking it and the model itself simpler as YOLOv11 had more updates and addition to it. The YOLOv11n-seg-DCA has a slightly higher complexity due to the addition of the DCA module.

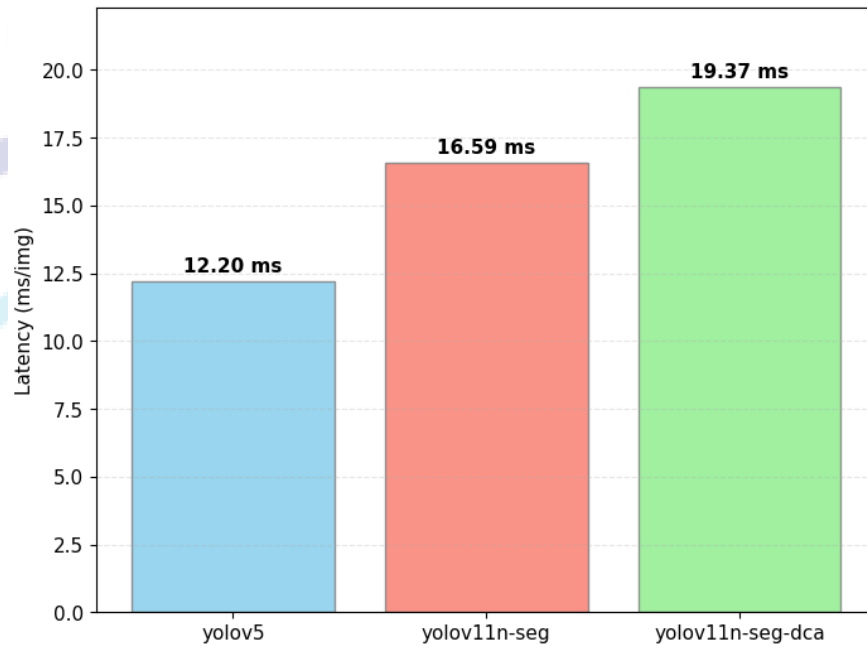


Figure 17 Object detection inference time comparison

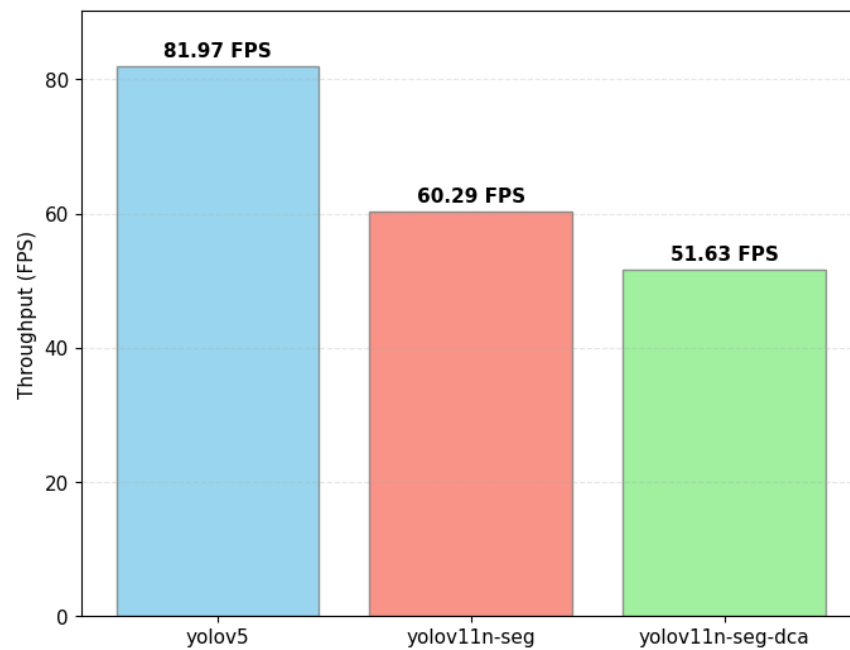


Figure 18 Object detection FPS comparison

To evaluate the viability of the detection models, end-to-end inference latency (ms/img) and throughput (Frames Per Second, FPS) were measured, as illustrated in Figure 17 and Figure 18, respectively. It should be noted that these specific benchmarks evaluate the detection architectures using standardized, preprocessed image inputs loaded directly into memory. Consequently, the reported speeds reflect the maximum theoretical performance of the detection stage in isolation. In a live diagnostic pipeline, the effective operational speed would be marginally lower due to the necessary overhead of I/O operations, such as reading raw microscopy files from storage, applying initial resolution scaling, and completing memory transfers to the GPU before inference can begin.

As expected, in line with the model complexity of the models, the legacy YOLOv5 architecture achieved the lowest processing latency at 12.20 ms per image, translating to a throughput of 81.97 FPS. However, this high speed is inherently tied to its architectural limitation of producing only bounding box coordinates without the computational burden of pixel-level instance segmentation with the model itself being simpler.

The baseline YOLOv11n-seg model, which introduces the mask prediction head required for instance segmentation, recorded a higher latency of 16.59 ms (60.29 FPS). The proposed YOLOv11n-seg-DCA model recorded a latency of 19.37 ms, yielding a throughput of 51.63 FPS. The integration of the Deformable Curvature Attention (DCA) module introduces a modest processing overhead of approximately 2.78 ms per image compared to the baseline.

Given that 51.63 FPS remains significantly above the standard real-time processing threshold (typically 30 FPS for continuous video or automated slide scanning), this slight reduction in throughput is a highly acceptable trade-off. The minor computational cost is well justified by the superior boundary delineation and overlapping specimen separation provided by the DCA module, ensuring that high-quality, accurately segmented crops are extracted for the subsequent classification stage.

4.2. Classification Model (RepVGG-CA)

4.2.1. Training

To assess the convergence stability and generalization capabilities of the proposed RepVGG-CA model, the training and validation were tracked for 80 epochs. The training and validation loss were shown in Figure 19, and the training and validation accuracy in Figure w20 as a function of the number of epochs.

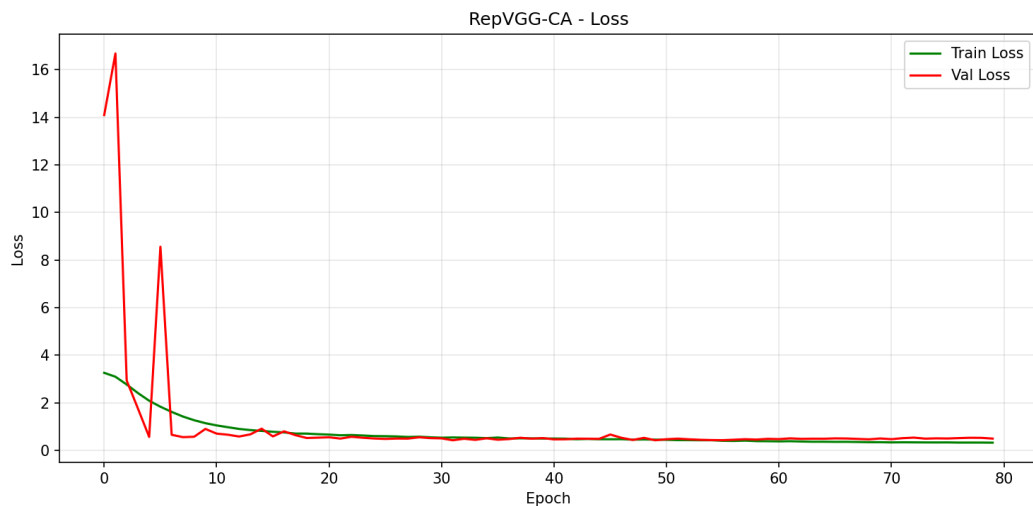


Figure 19 RepVGG-CA loss graph

The loss curves presented in Figure 19 confirm the observations derived from the accuracy trajectories and indicate convergence and stability of models. The training loss is initially 3.24, while the validation loss is initially much higher at 14.10. The validation loss has a very large peak at epoch 1 with a value of around 16.70. There's a second, smaller peak in validation loss at epoch 5, at around 8.50.

These severe spikes in the initial validation loss are typical of deep neural networks utilizing attention mechanisms on small datasets. Following epoch 10, the validation loss drops below 1.00 and stabilizes. Both the training and validation loss curves exhibit a smooth, asymptotic decline toward zero. By epoch 80, the training loss converges to 0.30, and the validation loss stabilizes at 0.48. The minimal discrepancy between the final training and validation loss values confirms that the optimization process was highly stable. The lack of divergence between the two curves in the later epochs indicates that the regularizing constraints of the compact design helped mitigate the overfitting trends that degrade performance on the limited dataset.

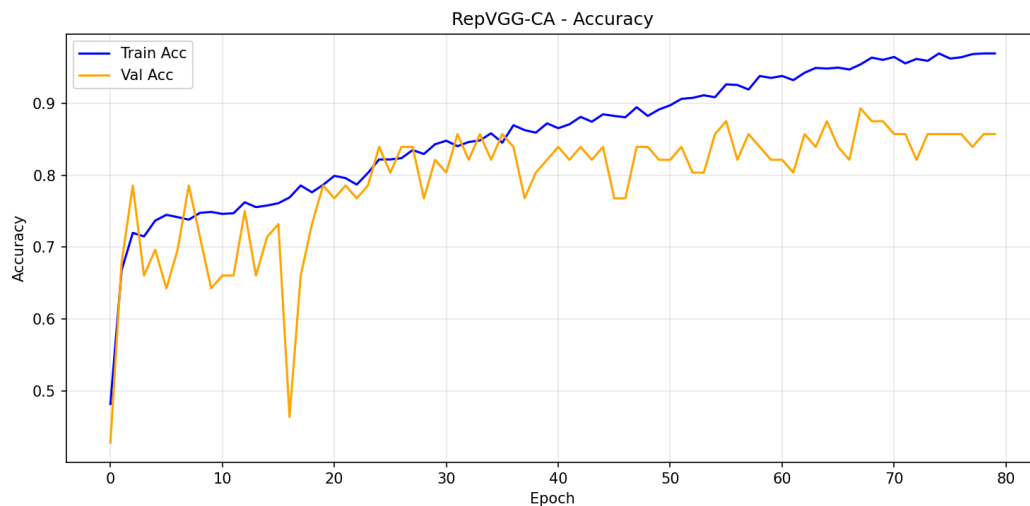


Figure 20 RepVGG-CA accuracy graph

The accuracy curves in Figure 20 show a fast initial learning rate and a slow steadying. The training accuracy is about 48.00%, and the validation accuracy is about 43.00% at the start of the training process (epoch = 0). Both of the accuracy metrics increase rapidly during the first five epochs, reaching a local maximum of about 78.50% at epoch 2 in the case of validation accuracy. This early peak is followed by a period of high volatility, with important fluctuations. There is a significant drop in validation accuracy at epoch 16, with a temporary performance of approximately 46.00%.

This early instability and strong drop around epoch 16 is likely due to the limited amount of data. In these first few steps, the weights of the model are continually being updated, and the optimization algorithm is determining a stable local minimum. The accuracy curves of training and validation show a steady rise after epoch 20 and reach a plateau after that.

Despite efforts made to prevent overfitting, it was observed during the final training stages that the validation accuracy began to decouple from the training loss. The training monitored the accuracy from the validation, and the model weights that provided the highest accuracy were stored at epoch 67 of 87.50% validation accuracy. The training accuracy continues to increase and reaches a final value of 96.91% for epoch 80. The accuracy of the validation set also improves with each epoch, demonstrating less variation in later epochs

and converging towards about 85.7% at the end of the epoch. The generalization gap between training and validation of 11.2% is controlled.

4.2.1. Result and Comparison

To explore potential performance gains, a hyperparameter optimization phase was conducted using Optuna. A 3-fold cross-validation strategy was employed to search the hyperparameter space. The primary goal of this is to fine-tune the parameters to potentially extract better generalization from the limited dataset. The optimization process tested multiple parameters, including the batch size (16 or 32), weight decay (ranging from 0.0004 to 0.01), base learning rate (ranging from 0.00001 to 0.001), warmup epochs (ranging from 5 to 20), and label smoothing factor (ranging from 0 to 0.15) compared to the original baseline configuration. The optimized parameters discovered by Optuna are detailed in Table 12 below.

Table 12 Parameter Summary for Tuned Parameters

Configuration	Value
Input Resolution	224 x 224 pixels
Training Epochs	80
Batch Size	16
Optimizer	AdamW
Weight Decay	0.008751
Base Learning Rate	0.000257
Warmup Epoch	12
Learning Rate Scheduler	Cosine decay after warmup
Label Smoothing	0.0759

Following the automated tuning process, the newly configured model was evaluated against the unseen 10% test dataset to measure the model's generalization capability. The overall accuracy comparison between the original, manually configured model and the Optuna-tuned model is presented in figure 21.

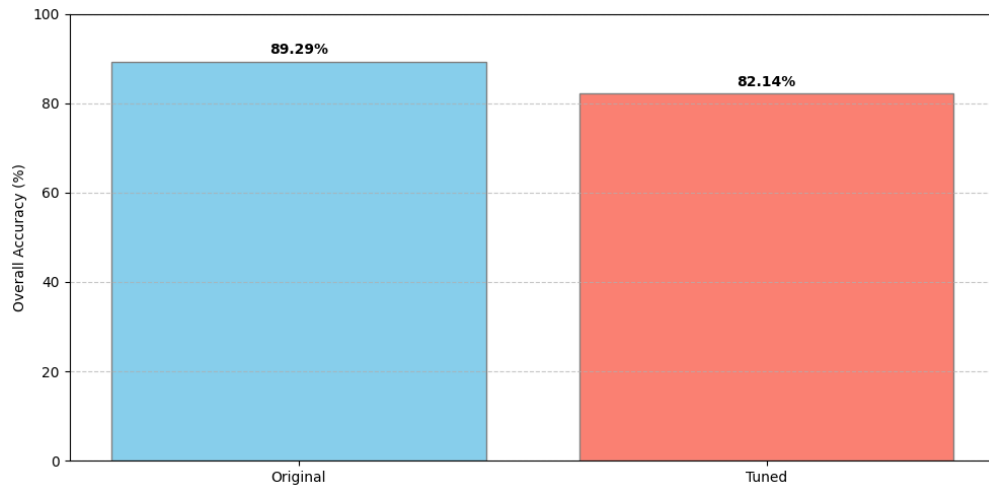


Figure 21 Test accuracy comparison original and tuned parameters

The test evaluation demonstrated that the automated optimization did not yield the anticipated improvements. In fact, the original configuration outperformed the tuned model by 7.15%. By aggressively optimizing to minimize the validation loss during the folds, the algorithm selected a parameter combination that ultimately crippled the model's adaptability on entirely unseen data. Specifically, the tuned weight decay of 0.008751 is significantly more penalizing than the original 0.005, and the label smoothing was sharpened from 0.1 down to 0.0759. This combination of stricter weight penalization and sharper target distributions excessively constrained the network. It prevented the model from capturing the natural, subtle morphological variations of the nematodes present in the test set. The manually selected original parameters provided a much more stable regularization boundary, proving to be more suitable for this classification task. Consequently, the original model configuration was retained as the primary architecture for the combined diagnostic pipeline.

To evaluate the performance of the RepVGG-CA architecture, the 10% test dataset was used. The key metrics used for this evaluation are: Accuracy, Precision, Recall, and F1 score. A comparative study was carried out between the proposed architectural improvements of the classification model and existing baseline models to validate the proposed model. The architectures used

for this benchmark are the EfficientNetV2B0 architecture from Shabrina et al. (2023) with the parameters mentioned in the paper. Also, the baseline RepVGG-A0 architecture, with the same parameters as the proposed RepVGG-CA.

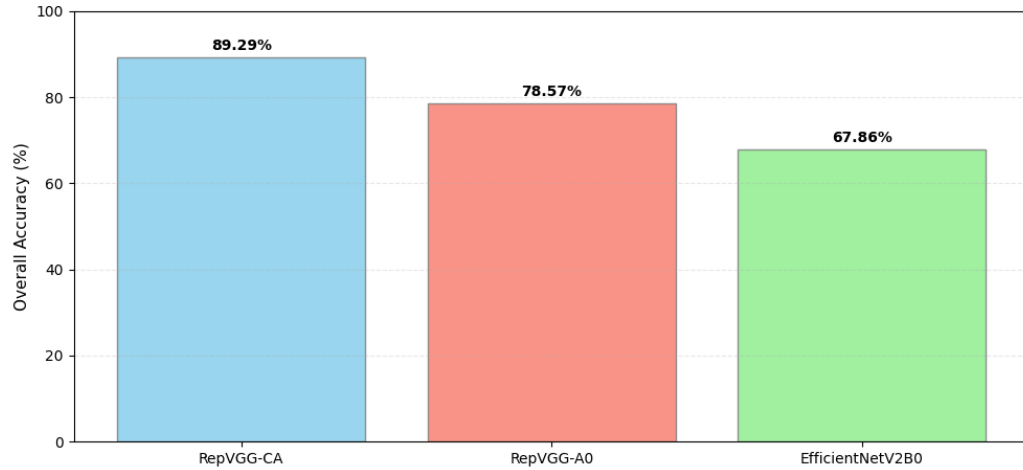


Figure 22 Test accuracy comparison of the models

The final classification results of the models evaluated are shown in Figure 22. The overall accuracy of the RepVGG-CA model is the highest with 89.29%. This outcome indicates that both a small model size design and the Coordinate Attention mechanism enable the system to detect relevant features in the small size of a nematode set. As a comparison, the accuracy of the RepVGG-A0 model was 78.57%. The RepVGG-CA variant is believed to outperform the base model by strategically applying $a=0.5$ and $b=1.5$ channel multipliers to the first 4 stages and the last stage, respectively. In addition, the fourth stage was shortened from 14 to 8 blocks. These structural changes were done specifically to reduce the high level of risk of overfitting that occurs when training with a small sample size. This reduces the number of parameters, and the model will not memorize the training noise, which will greatly enhance its ability to generalize in microscopic images.

EfficientNetV2B0 had the lowest performance with an accuracy of 67.86%. This shows 21.43% gain of the proposed RepVGG-CA compared to the benchmark architecture. The analysis of learning curves for EfficientNetV2B0 shows that there is a significant difference between the

training curve and validation curve. This difference is a clear sign of overfitting, which is often encountered when attempting to deploy complex architectures such as EfficientNet with small datasets where the number of samples is small, and the depth of feature hierarchies is not possible. The RepVGG-A0 model has a higher accuracy than EfficientNetV2B0, indicating that the RepVGG-based framework is more suitable for extracting relevant features from nematode samples than the EfficientNetV2B0 framework. The re-parameterizable multi-branch structure of RepVGG enables enhanced feature extraction for the micro texture, compared to the characteristics of EfficientNet's depth-wise separable bottlenecks.

The multi-branch topology of RepVGG trains with 3x3 convolutions, 1x1 convolutions, and identity mappings. The architectural diversity allows the model to represent a wider range of gradient information and morphological detail, even when data is limited. On the other hand, the MBConv blocks in the EfficientNetV2B0 are depth-wise separable convolutions, restricting the spatial correlation depth of a single layer. With a small dataset, the standard convolutional layers, such as the ones used in the EfficientNetV2B0, seem to be unable to properly capture the fine details and the textures of the nematode samples, as evidenced by the significant lead that RepVGG-CA has. Because of the use of fixed kernel sizes and standard pooling operations, many of the models have a lack of fine-grained spatial information, directly contributing to the overfitting problem seen in the baseline models.

The kernel with 3x3 dimensions, formed by the fusion of the RepVGG branches, offers better classification performance for Meloidogyne with high-frequency details and subtle anatomical variations. Moreover, the proposed model was able to overcome the shortcomings of conventional convolutional layers by incorporating positional information into the channel attention mechanism via Coordinate Attention. This enables the network to highlight and focus on regions that are important, like the tail and esophageal structures. The anatomical features are crucial to the identification of Meloidogyne

graminicola from *Meloidogyne incognita* in a small data environment. The 10.72% boost in the base RepVGG-A0 indicates that spatial awareness, combined with the small number of parameters, is an important mechanism to achieve high accuracy without overfitting on small-sized samples, which is evident in the more complex RepVGG-A0 and EfficientNetV2B0 configurations.

Table 13 *Meloidogyne Graminicola* Performance Metrics

Model Architecture	Precision	Recall	F1 Score
EfficientNetV2-B0	70.00%	82.35%	75.68%
RepVGG-A0 (Base)	92.31%	70.59%	80.00%
RepVGG-CA (Proposed)	100.00%	82.35%	90.32%

Observed in Table 13, the proposed RepVGG-CA model recorded a perfect precision score of 100.00% for the *Meloidogyne graminicola* class, so no false positives were recorded for this class. This is much better than the RepVGG-A0 (92.31%) and EfficientNetV2-B0 (70.00%). Good precision and recall of 82.35% led to the best F1 Score of 90.32%. The precision of EfficientNetV2-B0 was low, implying it was confused between species, this is probably because it was not able morphology with the small dataset. The RepVGG-A0 model exhibited high precision but with the lowest recall (70.59%), suggesting the model has high accuracy, but it missed a substantial amount of the true samples.

Table 14 *Meloidogyne Incognita* Performance Metrics

Model Architecture	Precision	Recall	F1 Score
EfficientNetV2-B0	62.50%	45.45%	52.63%

RepVGG-A0 (Base)	66.67%	90.91%	76.92%
RepVGG-CA (Proposed)	78.57%	100.00%	88.00%

The evaluation of *Meloidogyne incognita* performance in Table 14 further confirms the ability of the proposed architecture. The recall for the RepVGG-CA model with this species was 100.00%, which means it was able to correctly identify all of the instances of this species in the test set. The precision (78.57%) and F1 Score (88.00%) of this model are higher than those of the baseline architectures. EfficientNetV2-B0 showed a critical failure in this classification with a recall of 45.45%, implying it was not able to identify the morphological characteristics of *M. incognita* well. RepVGG-A0 had a recall of 90.91%, which is higher than EfficientNet; however, it was still not as high as the proposed model.

It can be seen from the tables that the Coordinate Attention module is effective, and the RepVGG-CA model performs better than the other two models in both tables. The model is finely tuned to the positional features required for species differentiation, thereby eliminating the sporadic drops in performance seen in EfficientNet and the uneven precision-recall compromises seen in the RepVGG-A0.

The major difference in performance for the two species is due to the class imbalance in the original dataset. *Meloidogyne graminicola* is the majority class, and *Meloidogyne incognita* is the minority class. To overcome this, there was a protocol that was introduced during the data preprocessing stage to deliberately decrease and limit the amount of augmented data that was generated for the majority class. Even after applying this protocol, the greater feature variance and structural representation inherent to the larger native pool of *Meloidogyne graminicola* images continued to exert a strong influence on the decision boundaries of the network. This residual imbalance effect is a possible reason for the perfect 100.00% precision of the proposed model in the

case of *Meloidogyne graminicola* and the lower precision of 78.57% in *Meloidogyne incognita*.

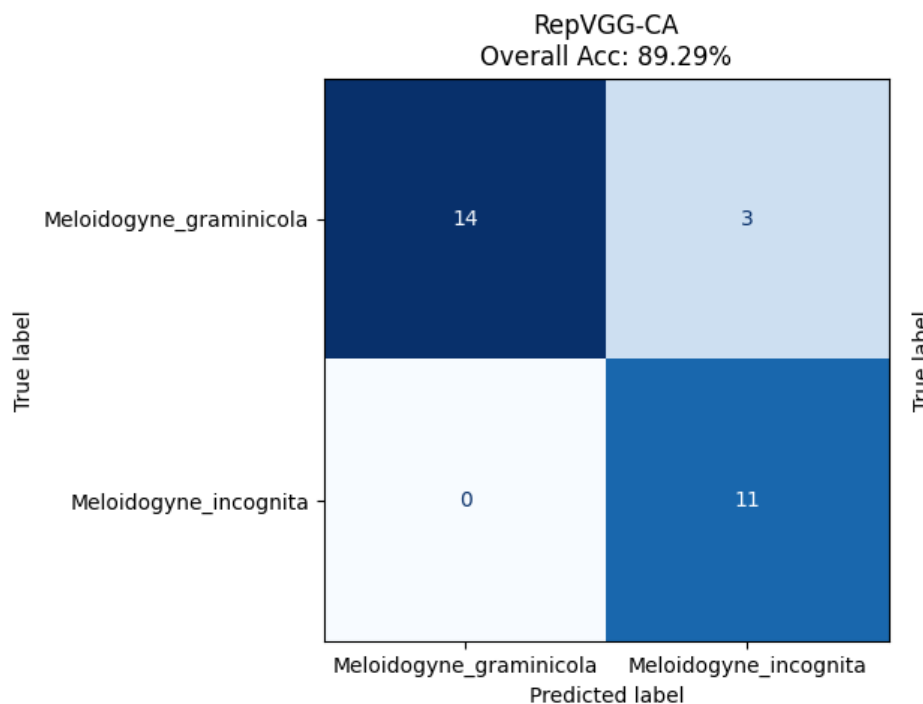


Figure 23 RepVGG-CA confusion matrix

The performance of the proposed RepVGG-CA model is shown in Figure 23. It demonstrates a correct classification of 14 of 17 cases of *M. graminicola* and an incorrect classification of 3 cases in the class *M. incognita*. Importantly, none of the 11 true samples of *M. incognita* were misidentified, while no false negatives were found in the minority class. This is consistent with the previous report for high recall of *M. incognita* and high precision of *M. graminicola*. The error profile shows that the model is conservative in its identification of the majority class, only misclassifying samples into the minority category rather than failing to recognize the presence of the minority species itself.

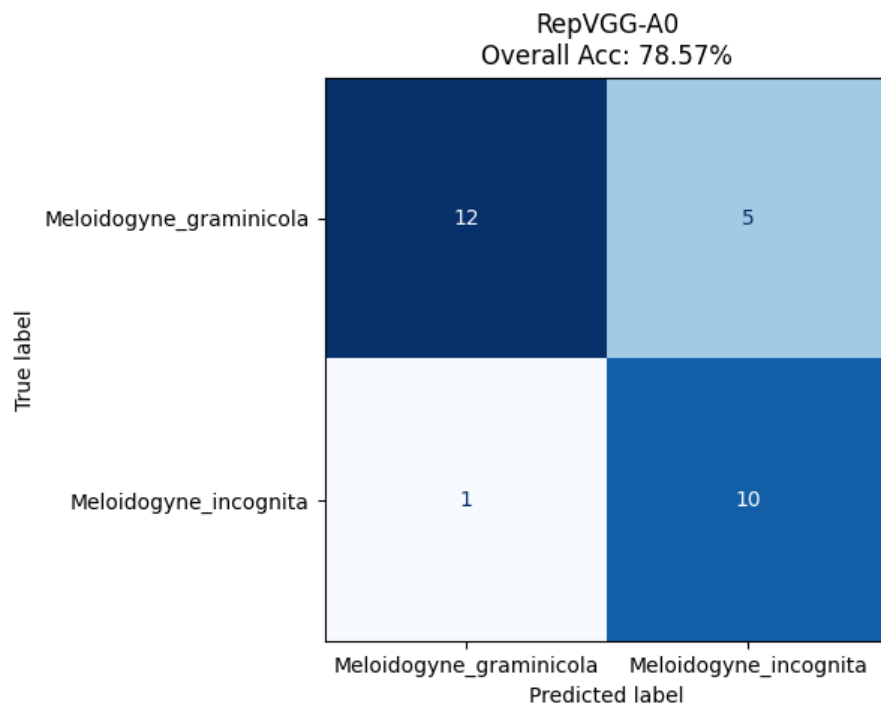


Figure 24 RepVGG-A0 confusion matrix

RepVGG-A0's confusion matrix in Figure 24 shows a significant increase in confusion. It correctly identified 12 out of 17 *M. graminicola* samples, but misclassified 5 as *M. incognita*. In addition, it had 1 false negative for *M. incognita*. This error pattern is indicative of the lack of discriminative focus given by the Coordinate Attention mechanism. The attention module is crucial for the model to distinguish between the fine anatomical details of the two species; without the spatial weighting, the model would have to integrate a larger region, making its classification less accurate and resulting in higher entropy in its decision-making.

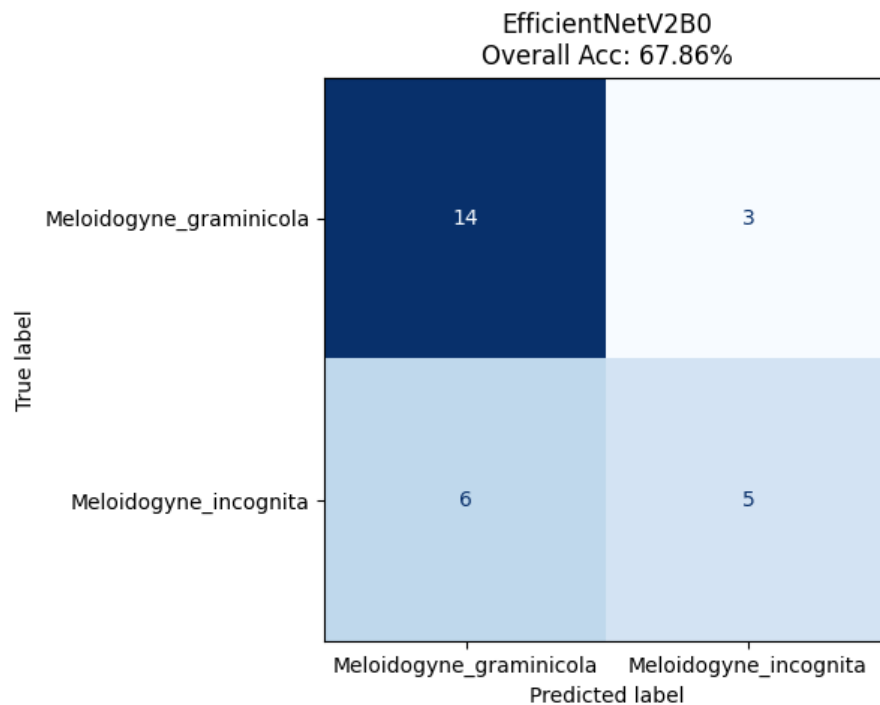


Figure 25 EfficientNetV2B0 confusion matrix

The efficiency of the model can be seen in the confusion matrix shown in Figure 25. The degradation is the most pronounced when it comes to EfficientNetV2B0. It correctly identified 14 *M. graminicola* samples, but misdiagnosed 6 of the 11 specimens of *M. incognita* as the majority sample. There is a significant majority class bias and a low recall of 45.45% on *M. incognita*. The matrix appears to show that EfficientNetV2B0 is essentially overfitting on a small, imbalanced dataset, as it seems to have collapsed towards the majority class features. The proposed RepVGG-CA relied on structural pruning and attention to balance the perspective of both classes; however, the standard EfficientNet structure failed to learn a good internal representation for the minority class. These findings confirm the critical importance of architectural modifications in RepVGG-CA for successful nematode species identification in limited data scenarios.

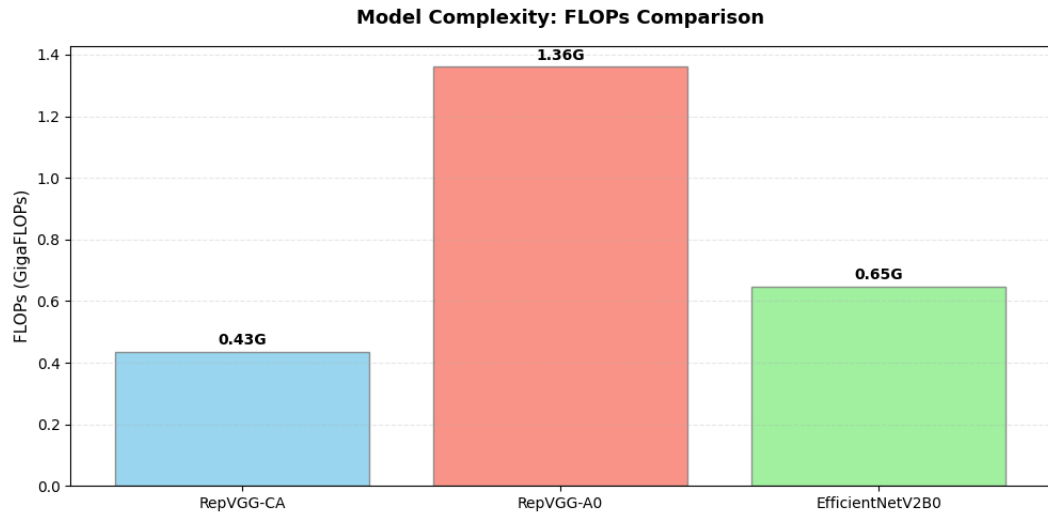


Figure 26 Classification model complexity

The computational complexity of the classification models, measured in GigaFLOPs (GFLOPs), seen in Figure 26, further highlights the efficiency of the proposed architecture, as illustrated in the figure above. Among the compared models, the proposed RepVGG-CA has the lowest number of GFLOPs in inference (0.43). The unoptimized RepVGG-A0 baseline model, on the other hand, uses 1.36 GFLOPs. The direct result of the targeted structural pruning is this reduction in model complexity, with 68.4% of the complexity reduced. In particular, the decrease in the stage 4 depth and optimization of channel width multipliers. The efficient model, EfficientNetV2B0, requires 0.65 GFLOPs, generally considered an efficient model in most benchmarking. The proposed RepVGG-CA is thus approximately 33.8% lighter than EfficientNetV2B0 and has significantly higher accuracy (89.29% vs 67.86%).

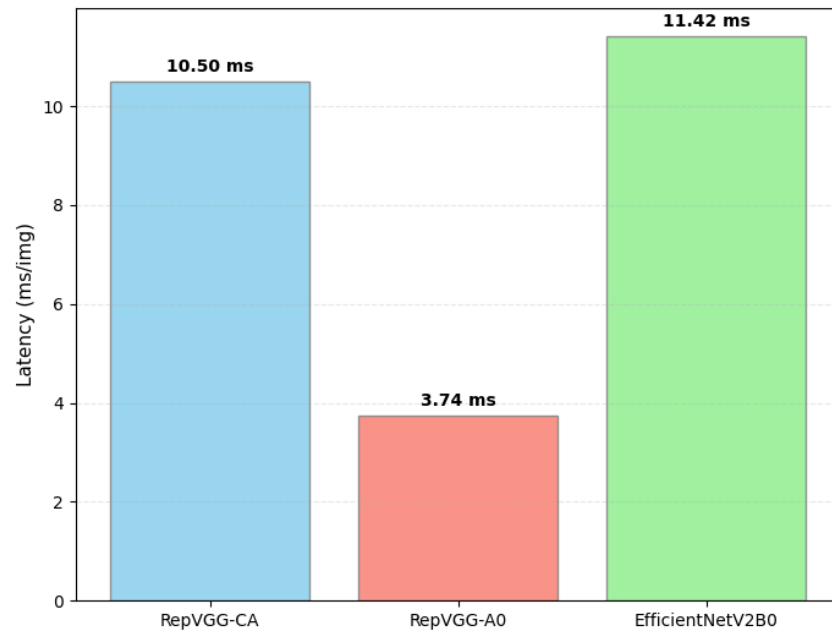


Figure 27 Classification inference time comparison

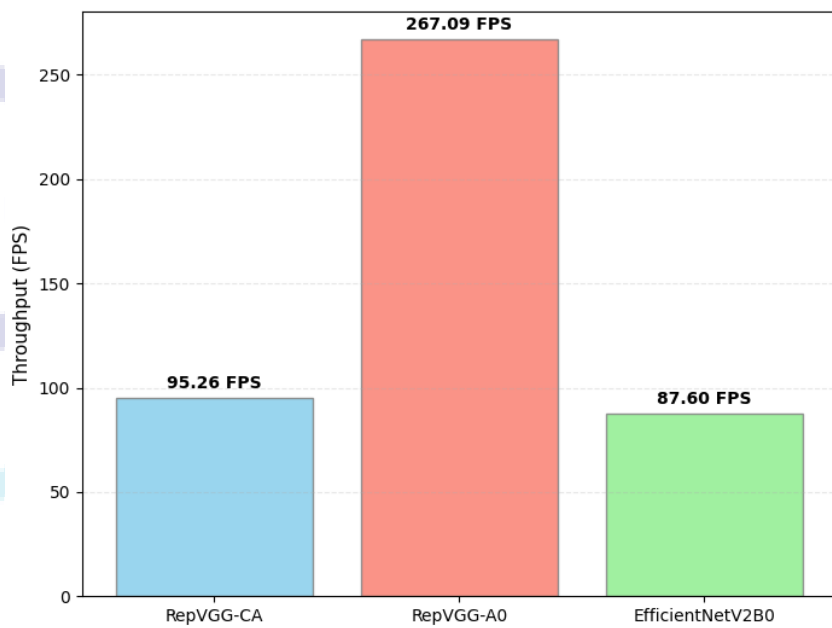


Figure 28 Classification FPS comparison

The inference latency and throughput for the isolated classification stage were similarly benchmarked to ensure downstream efficiency, as detailed in Figures 27 and 28, respectively. It is important to note that these specific

benchmarks evaluate the classification architectures using preprocessed, isolated image crops of individual nematodes. Consequently, the reported speeds reflect the theoretical performance of the classification stage in isolation. This isolated processing is inherently faster than the operational speed of the complete diagnostic pipeline, which must first process complex, full-field microscopic images through the detection stage before extracting and passing the cropped Regions of Interest (ROIs) to the classifier.

With these isolated test setups, baseline RepVGG-A0 outperformed the other models with a throughput of 267.09 FPS and a latency of 3.74 ms when processing pre-cropped images. This speed is inherent to the base RepVGG architecture that exploits structural re-parameterization at test time to fully collapse its multi-branch training topology to a highly optimized single-path convolutional network with 3x3 convolutions.

The proposed RepVGG-CA architecture achieved a latency of 10.50ms, and throughput of 95.26 FPS. The inference time gain is a natural architectural effect, as the Coordinate Attention (CA) module adds extra spatial pooling operations and sigmoid activations for calibration of feature maps. These dynamic attention modules differ from regular convolutions as they can't be completely re-parameterized into static weights.

However, the proposed RepVGG-CA model is still more computationally efficient than the fastest and the benchmark EfficientNetV2B0 model, with a latency of 11.42 ms (87.60 FPS) in the group. Although some overhead is induced by the attention mechanism, the RepVGG-CA model still achieves a high-speed throughput of almost 100 FPS on preprocessed crops. This validates that the system could be trained to balance high-precision morphological feature extraction with the desired fast operation time without becoming a computational bottleneck during training of the classification stage of the diagnostic pipeline.

4.3. Combined Pipeline

The proposed pipeline is a multi-stage pipeline based on object detection and image classification. This system uses the Yolov11-seg-DCA object detection model to detect the location and determine nematode bounding boxes in the images. The cropped localized regions are then sent to the proposed classification model (RepVGG-CA) for species identification. The pipeline is applied to 10% test set, which was not seen by either model, with 25 images and 29 nematodes.

The first step of the pipeline is to separate each nematode from the rest of the microscopic image using the Yolov11-seg-DCA model. The metrics acquired are comparable to those of Yolov11-seg-DCA (precision = 96.67% with 100% recall), in which all 29 nematode instances are acquired with only one false positive. Next, the output of the Yolov11-seg-DCA model is used to prepare the input and given to the RepVGG-CA classification model. Of the 29 real crops passed to the classifier, 26 were assigned the correct species label. With one false positive and 29 true crops, this results in an accuracy of 86.67% seen in Table 15.

Table 15 Pipeline classification result

Metric	Value
False positive crops	1
True positive crops	29
Crops Labeled Correctly	26
Classification Accuracy	86.67%

The end-to-end image-level classification success is used as the efficiency measure for the overall integrated pipeline. This evaluation takes into consideration the misclassifications of the second stage and the false positive detection of the first stage. The full pipeline metrics are shown in Table 13.

Table 16 Full pipeline results

Accuracy	Precision	Recall	F1 Score
88%	87%	88%	87%

The total accuracy of the integrated pipeline was 0.8800 on the image level. This rate represents successful detection and classification in all 22 images of the original 25 microscopic evaluation images. For each of these 22 successful images, the YOLO model successfully detected all nematode instances and produced bounding boxes, which were then cropped, and then the correct species designation was labeled by the RepVGG-CA classification model.

The precision, recall, and F1 score are recorded at 0.8667, 0.8819, and 0.8727, respectively. These metrics are indicative of the pipeline's performance with typical distributions of the species. The three images that did not succeed are explicitly blamed on the propagation of errors at a local scale. To be specific, two images were found to be confused, that is, the actual *M. graminicola* was misclassified by the classifier. Only one image did not reach the ideal success rate because the false positive YOLO detector bound a non-nematode background artifact, which was then given a species label by the closed set classifier.

This pipeline precision value is 86.67%, meaning that if a pipeline classifies a specimen as a certain nematode species, there is an 86.67% chance that the classification is correct. This value reflects the impact of the one false positive from the YOLO detector and the three classification errors in the real world. The pipeline is constrained to classify each crop, and the false detection directly affects the reliability of the prediction in *Meloidogyne incognita*, which demonstrates the high reliability of the final outputs, but is sensitive to localization noise in the early stage.

The pipeline recall of 88.19% indicates that the system is able to successfully capture and correctly identify 88.19% of all actual nematode specimens physically present in the field of view. This measure is bounded by

the four total classification misses, with a detection model that had a perfect recall of 100.00% by finding all true specimens. The ultimate recall suffers from the class imbalance noted earlier (where the majority class features caused a shift in the decision boundary of the classifier).

The pipeline F1 score of 87.27% indicates a balanced operational stability of the integrated system. This value shows that the system is able to effectively control the balance between sensitivity and specificity, in a small dataset with class imbalance and with no background filter. An F1 score of 87.27% is to be expected as the model does not have a high number of major diagnostic errors, such as over-reporting the majority species at the expense of the critical minority species.

Table 17 Pipeline inference time

Phase Stage	Latency
Stage 1: YOLOv11-seg-DCA detection (per image)	74.29 ms
Stage 2: RepVGG-CA classification (per crop)	42.01 ms
Stage 1+2 (per image)	138.14 ms

Table 18 Pipeline Troughput

Phase Stage	Troughput
Stage 1: YOLOv11-seg-DCA detection (per image)	13.46 FPS
Stage 2: RepVGG-CA classification (per crop)	23.8 FPS
Stage 1+2 (per image)	7.24 FPS

The operational efficiency of the integrated pipeline was evaluated to assess its readiness for practical deployment. Table 17 details the latency while table 18 details the troughput of the complete pipeline when processing raw microscopic data.

The execution of the combined pipeline is broken down into two distinct inference stages alongside the necessary data-handling operations. Stage 1, which employs the YOLOv11n-seg-DCA model for detection and instance

segmentation, recorded an average latency of 74.29 ms per image, yielding a throughput of 13.46 Frames Per Second (FPS). Stage 2, utilizing the RepVGG-CA model to classify the extracted nematode crops, recorded an average latency of 42.01 ms per crop, yielding a throughput of 23.8 Frames Per Second (FPS).. The total end-to-end latency to process a single microscopic image through the entire system amounted to 138.14 ms, yielding a full pipeline throughput of 7.24 Frames Per Second (FPS).

It is critical to note that these operational speeds are substantially slower than the theoretical, isolated benchmarks presented in Sections 4.1.2 and 4.2.2. This reduction in throughput is an expected consequence of deploying a fully integrated diagnostic pipeline. Unlike the isolated benchmarks, which evaluated the models using standardized, pre-loaded tensor inputs directly in memory. This end-to-end evaluation accounts for all requisite preprocessing and mid-pipeline data transformation.

The 138.14 ms total latency encompasses the time required to read raw microscopic images from storage, apply initial resolution scaling, and execute memory transfers to the GPU. Furthermore, the transition between Stage 1 and Stage 2 introduces significant computational overhead. The system cannot simply pass data from one model to the next; it must calculate the predicted mask boundaries, dynamically crop the Regions of Interest (ROIs) from the original high-resolution image, apply letterbox padding to preserve morphological aspect ratios, resize the newly cropped tensors to 224x224 for the classifier, and manage intermediate data transfers.

While a throughput of 7.24 FPS remains highly practical for the intended agricultural application. Microscopic soil sample analysis is fundamentally a discrete, slide-by-slide process rather than a continuous live-feed operation. Executing a complete diagnostic cycle, including multi-nematode localization, morphological extraction, and species-level classification. In just 138.14 milliseconds represents a massive acceleration over traditional, manual nematology workflows.



CHAPTER V

CONCLUSION

5.1. Conclusion

This study succeeded in developing and testing a 2-stage deep learning system, detecting and identifying the root-knot nematode at the species level, directly from unprocessed microscopic images. The pipeline can be used in the agricultural diagnostics domain, which is characterized by good localization abilities in object detection and fine-grained discrimination in image classification.

In the first phase of the pipeline, the proposed YOLOv11n-seg-DCA architecture was used. The model proved to be very effective for nematode detection in complicated microscopic backgrounds. The model achieved a 99.30% mAP50 (box). The performance is slightly below the initial 99.5% research objective, but the Deformable Curvature Attention module has done a great job in body segmentation and capture of curvilinear boundaries. During extraction, the recall rate of the model was 100.00%, which means that there were no specimens missed.

The second stage pipeline involved the *Meloidogyne graminicola* and *Meloidogyne incognita* classification model RepVGG-CA. The overall accuracy of the classifier was 89.29%. This outcome was not as high as the target accuracy of 98% set at the beginning of the study, but it is still very notable because of the small number of images (just 245) on which the results were based. The RepVGG-CA architecture achieved a significant increase in accuracy compared to baseline models, with an improvement of more than 10% over the RepVGG-A0 model and over 21% over the EfficientNetV2B0 model. This led to a substantial reduction in severe overfitting and attention to important spatial landmarks, thanks to the inclusion of Coordinate Attention and the decrease in the number of parameters.

The pipeline was then integrated and found to be accurate at the image level, with 22/25 images being processed correctly, for a final image-level accuracy of 88.00%. The overall system achieved a precision of 86.67%, a recall of 88.19%, and an F1 of 87.27%. The metrics showed that the system achieves the desired balance between sensitivity and specificity, and is a reliable automatic system that minimizes the work of microscopic analysis.

5.2. Suggestions

The results of this study suggest a number of areas of future research and optimization that would increase the reliability of nematode diagnosis. The main challenge faced while training the model was the small data size. The next version of this project should focus on increasing the database of microscopic images. A larger and more diverse collection of nematode samples would be directly helpful to the classification model in achieving greater accuracy for classification. A larger dataset would enable a deeper hierarchy of features to further decrease the generalization gap.

In addition, special attempts should be made to deal with the class imbalance in the data. The difference in performance between the majority class and the minority class shows that the decision boundaries of the classifier are still skewed. More physical samples of *Meloidogyne incognita* should be collected by researchers to balance feature variance between species. Alternatively, more sophisticated synthetic data generation methods, like Generative Adversarial Networks, may be looked into to artificially harmonize the training distributions.

It is also strongly recommended to test an integrated pipeline in real deployment scenarios. The current models were trained and tested with static and collected data sets. Validation of the diagnostic robustness of the pipeline would be a rigorous process if the pipeline were deployed with field samples of soil collected from different sites and processed with varying levels of soil residue, lighting, and microscope magnifications.

Finally, the scope of the classification model should be broadened to include additional nematode species. Expanding the architecture to recognize a wider variety of plant-parasitic nematodes, as well as distinguishing them from beneficial free-living nematodes, would vastly increase the practical utility of the pipeline for global agricultural stakeholders and integrated pest management systems.



REFERENCES

- Afzal, A., & Mukhtar, T. (2024). Revolutionizing nematode management to achieve global food security goals - An overview. *Heliyon*, 10(3), e25325.
- Bishop, C. M. (2006). *Pattern recognition and machine learning*. Springer.
- Coyne, D. L., Nicol, J. M., & Claudius-Cole, B. (2014). *Practical plant nematology: a field and laboratory guide*. International Institute of Tropical Agriculture (IITA).
- Ding, X., Guo, Y., Han, G., & Ding, X. (2021). RepVGG: Making VGG-style convnets great again. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*.
- Ding, X., Zhang, X., Ma, N., Han, J., Ding, G., & Sun, J. (2021). RepVGG: Making VGG-style ConvNets Great Again. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 13733–13742.
- Dutta, T. K., & Phani, V. (2023). The pervasive impact of global climate change on the plant-nematode interaction continuum. *Frontiers in Plant Science*, 14, 1143889.
- Eisenback, J. D. (1985). Diagnostic characters for identifying the four most common species of root-knot nematodes (*Meloidogyne* spp.). In *An advanced treatise on Meloidogyne* (Vol. 1, pp. 95-112). North Carolina State University Graphics.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.
- He, L., et al. (2025). Application of the YOLOv11-seg algorithm for AI-based landslide detection. *Scientific Reports*.
- Hung & Carpenter Hung, J., & Carpenter, A. (2017). Applying Faster R-CNN for object detection on malaria images. *2017 IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 56–61.

- Hou, Q., Zhou, D., & Feng, J. (2021). Coordinate attention for efficient mobile network design. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*.
- Jones, J. T., Haegeman, A., Danchin, E. G. J., Gaur, H. S., Helder, J., Jones, M. G. K., ... & Opperman, C. H. (2013). Top 10 plant-parasitic nematodes in molecular plant pathology. *Molecular Plant Pathology*, 14(9), 946-961.
- Kervadec, H., et al. (2021). Boundary loss for highly unbalanced segmentation. *Medical Image Analysis*.
- Khanam, R., & Ahmed, M. (2024). YOLOv11: An overview of the key architectural enhancements. *arXiv preprint arXiv:2410.17725*.
- Lin, T. Y., Goyal, P., Girshick, R., He, K., & Dollár, P. (2017). Focal loss for dense object detection. *Proceedings of the IEEE International Conference on Computer Vision*, 2980–2988.
- Loshchilov, I., & Hutter, F. (2017). SGDR: Stochastic gradient descent with warm restarts. *International Conference on Learning Representations*.
- Loshchilov, I., & Hutter, F. (2019). Decoupled weight decay regularization. *International Conference on Learning Representations*.
- Mitchell, T. M. (1997). *Machine learning*. McGraw-Hill.
- More, B., & Bhosale, S. (2023). A comprehensive survey on object detection using deep learning. *Revue d'Intelligence Artificielle*, 37(2), 407–414.
- Oerke, E. C. (2006). Crop losses to pests. *The Journal of Agricultural Science*, 144(1), 31-43.
- Oliveira, C. M. G., Monteiro, A. R., & Blok, V. C. (2011). Morphological and molecular diagnostics for plant-parasitic nematodes: Working together to get the identification done. *Tropical Plant Pathology*, 36(2), 65–76
- OpenCV. (2025). *Laplace Operator - OpenCV Documentation*.
- Pun, T. B., Neupane, A., Koech, R., & Walsh, K. (2023). Detection and counting of root-knot nematodes using YOLO models with mosaic augmentation. *Biosensors and Bioelectronics: X*, 15, 100407.

- Redmon, J., & Farhadi, A. (2018). YOLOv3: An incremental improvement. *arXiv preprint arXiv:1804.02767*.
- Shabrina, N. H., Lika, R. A., & Indarti, S. (2023). Deep learning models for automatic identification of plant-parasitic nematodes. *Artificial Intelligence in Agriculture*, 7, 1-12.
- Shah, M. M., & Mahamood, M. (Eds.). (2017). *Nematology - concepts, diagnosis and control*. InTech. doi: 10.5772/66851
- Shan, Z., et al. (2025). RSM-YOLOv11: Lightweight steel surface defect segmentation network. *IEEE Access*.
- Sikora, R. A., & Fernandez, E. (2005). Nematode parasites of vegetables. In *Plant parasitic nematodes in subtropical and tropical agriculture* (pp. 319-392). CABI Publishing.
- Simonyan, K., & Zisserman, A. (2014). Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*.
- Szeliski, R. (2022). *Computer vision: Algorithms and applications* (2nd ed.). Springer.
- Ultralytics. (2025). *Ultralytics YOLO11*.
- Van Den Hoogen, J., Geisen, S., Routh, D., Ferris, H., Traunspurger, W., Wardle, D. A., & Crowther, T. W. (2019). Soil nematode abundance.
- Wang, Y., Zhang, L., & Li, X. (2024). Comparing YOLOv11 and YOLOv8 for instance segmentation of immature green apples. *arXiv preprint arXiv:2410.19869*.
- Zhang, T., Wang, H., Li, J., & Alotaibi, S. R. (2021). Deep learning for object detection: A survey. *Computer Systems Science & Engineering*, 38(2), 199–220.
- Zheng, Z., Ge, Z., Yang, X., Liu, X., Qin, L., Wang, X., & Zhou, Y. (2024). A-RepVGG: Research on classification algorithms based on deep learning and wood CT images. *Research Square*.

BIOGRAPHY AUTHOR



Evan Aditya Chandra is a student in the Computer Science program within the Faculty of Engineering and Technology at Sampoerna University. My educational background provides a foundation in core computing principles, software development, and systems architecture. I am deeply motivated by the potential of technology to solve complex problems and streamline operational processes across various industries. Throughout my academic journey, the focus has remained on bridging theoretical computing concepts with practical software applications.

